

Leveraging Lineage Barcodes as Natural Augmentations for Contrastive Learning of Cell Fate in scRNA-seq Data

Shizhao Joshua Yang¹ Yixin Wang² Kevin Z. Lin³

Abstract

Deciphering how cells commit to future fates is essential for developing precision therapeutics that can reprogram stem cells or modulate immune functions. However, isolating these fate-determining signals in single-cell lineage tracing (scLT) remains challenging because differentiation programs are often confounded by unrelated processes like the cell cycle. To address this, we introduce Lineage-aware Contrastive Learning (LCL), a framework that treats inheritable lineage barcodes as a "natural" data augmentation to isolate subtle, lineage-specific signals. LCL utilizes a semi-supervised architecture to align unlabeled cells, facilitating the transfer of lineage structures to clinical datasets where explicit barcoding is unavailable. We demonstrate LCL's utility by predicting future cell-type compositions from early-time points, effectively modeling longitudinal fate commitment from cross-sectional data. Benchmarking on hematopoietic and fibroblast systems shows that LCL outperforms standard models like scVI, establishing contrastive learning as a scalable paradigm for understanding and potentially manipulating cellular differentiation.

1. Introduction

Understanding how cells commit to their future fates – the transition from an initial state to specific final states – enables therapeutics to precisely manipulate undifferentiated cells. Such knowledge is critical for improving immunotherapies (Sugiura et al., 2022) and reprogramming protocols (Sharma et al., 2022). However, the destructive nature of

single-cell sequencing prevents direct longitudinal study of individual cells. To overcome this, single-cell lineage tracing (scLT) protocols infect cells with inheritable lentiviral barcodes; as cells divide, their progeny inherit these markers, allowing scientists to track ancestry relationships over time (Chen et al., 2022a; Jones et al., 2023). These protocols simultaneously sequence gene expression and lineage barcodes, providing richer context than standard single-cell RNA-sequencing (scRNA-seq) (Wagner & Klein, 2020). The resulting combination of temporal and clonal information offers unprecedented insights into cell fate dynamics (Fig. 1). However, existing computational methods struggle to isolate subtle, fate-determining signals from other biological confounding processes.

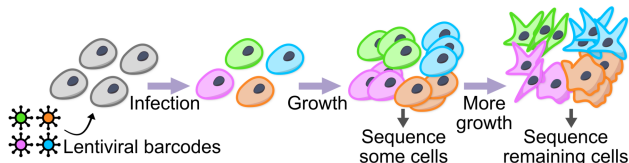


Figure 1. Schematic of single-cell lineage barcoding protocol.

Many methods have been developed to model scRNA-seq data and understand fate commitment without using lineage barcodes (Trapnell et al., 2014; Street et al., 2018; Setty et al., 2019; Bergen et al., 2020; Chen et al., 2022b). However, all these approaches share a fundamental obstacle. A cell's gene expression is a composite of many mechanisms that occur simultaneously, not all of which are related to the lineage-specific developmental programs of interest. For example, gene expression gives only partial information about the cell cycle, cell identity, and various signaling mechanisms. These factors all confound the subtle signals that indicate differentiation and future fate (Pascual-Ahuir et al., 2020; Eisenstein, 2020; Nitzan & Brenner, 2021). Although lineage barcodes (i.e., labels) reveal critical biological findings that are statistically invisible to standard methods, the vast majority of currently available single-cell datasets lack these explicit markers. To address this, we develop Lineage-aware Contrastive Learning (LCL). LCL is a semi-supervised deep learning method with a contrastive architecture. It treats lineage labels as "natural" data augmentations to isolate fate-determining signals. Crucially, LCL

¹Dartmouth College, Hanover, NH 03755, USA ²University of Michigan, Ann Arbor, MI 48109, USA ³University of Washington, Seattle, WA 98107, USA. Correspondence to: Shizhao Joshua Yang <shizhao.yang.gr@dartmouth.edu>, Kevin Z Lin <kzlin@uw.edu>.

enables transfer of learned lineage labels from scLT datasets to non-barcoded scRNA-seq datasets. By aligning the embedding spaces of both datasets, LCL helps researchers model cellular fates in typical scRNA-seq studies. This is especially useful where in vitro barcoding is unavailable.

2. Related work

The challenge in isolating the lineage signals that drive different cell fates is that these signals are often overshadowed by more prominent signals, such as cell-type variation, which may be orthogonal to cell fate in complex biological systems. See Fig. 2 for the schematic of scLT data. Many current methods in single-cell analysis use variational autoencoder (VAE) architectures to model single-cell data (Lopez et al., 2018; Choi et al., 2023; Weinberger et al., 2023; Boyeau et al., 2023), which excel at reconstructing gene expression. However, since these methods are unsupervised, the resulting representations may not sufficiently isolate fate commitment signals from other biological processes in the cell, making it difficult to study how cells commit to their fates.

	p genes	Lineage label	Time point	Cell type
Cell 1	Log-normalized gene expressions	Lineage 1	Day 4	Initial
Cell 2		Lineage 1	Day 12	Final state 1
Cell 3		Lineage 2	Day 4	Initial
$\vdots$		$\vdots$	$\vdots$	$\vdots$
Cell n		Lineage 2	Day 12	Final state 2

Figure 2. Schematic of a scLT dataset containing gene expression vectors, lineage, time point, and cell type of each cell.

Since we want to model the lineage barcodes, our problem is *supervised*. On the other hand, standard deep learning prediction methods are also not suitable for learning how gene expression encodes fate programs because the signal-to-noise ratio is too low. For instance, using VariancePartition (Hoffman & Schadt, 2016) to quantify how much measured covariates explain the variability of each gene’s expression, we see that while some genes strongly encode a cell’s lineage, the vast majority of genes are dominated by variability in cell type or time point (Fig. 3). Therefore, we deploy contrastive learning to uncover the faint gene expression signals specific to lineage information. Contrastive learning learns an embedding in which cells in the same lineage are close to each other if and only if they are in the same lineage. This idea originates from the computer vision literature to embed images where *data augmentation* mechanisms (such as rotations, cropping, etc.) of individual images, which do not fundamentally change the content of the image, help the deep learning model to learn more generalizable repre-

sentations (Chen et al., 2020; Chuang et al., 2020; Khosla et al., 2020). By treating lineage barcodes as a naturally occurring data augmentation mechanism, we encourage the deep learning model to hone in on gene expression components specific to lineage information. In what’s to come, we compare our proposed method to CoSpar (Wang et al., 2022) and GEMLI (Eisele et al., 2024), two other methods that specifically incorporate the lineage barcodes into their statistical modeling. See Appendix B for a further discussion of related work.

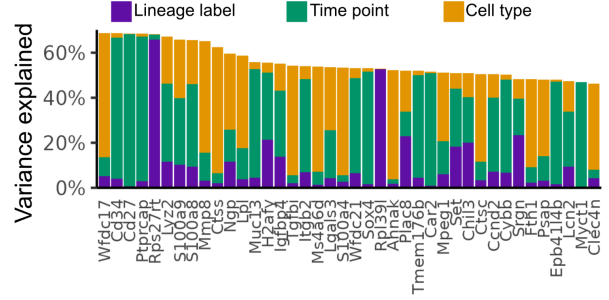


Figure 3. The amount of variance explained by the lineage label, time point, and cell type among the 40 genes with the greatest variance in our hematopoietic differentiation system (Weinreb et al., 2020). Most gene expression variability is explained by cell type or time point, not by lineage labels.

3. Method

3.1. LCL’s semi-supervised contrastive learning architecture and loss function

The primary objective of Lineage-aware Contrastive Learning (LCL) is to learn a representation space that prioritizes lineage fate-determining signals over dominant transcriptional noise. Given a dataset $\{(x_i, y_i)\}_{i=1}^n$ of n cells with p -dimensional gene expression x_i and lineage labels (i.e., barcodes) y_i , we learn an embedding z_i that disentangles fate-relevant biological programs from confounding processes. Crucially, LCL is not a classifier; its goal is not to predict a cell’s lineage label y_i , but rather to learn an embedding that reveals the “fate commitment” of cells as they transition from an initial state to specific final states, even in datasets where lineage barcodes were never explicitly recorded. We optimize the embedding according to the following three goals:

- **Goal #1:** Cluster cells by lineage labels to isolate subtle, fate-determining signatures from confounding sources of variation.
- **Goal #2:** Capture generalizable latent features that allow cells from unseen lineages to naturally cluster by ancestry in the embedding space, despite the model having no prior exposure to those specific labels.
- **Goal #3:** Facilitate the mapping of non-labeled

datasets into this lineage-aware embedding, enabling the discovery of lineage fate dynamics in studies where *in vitro* lineage barcoding is impossible.

To achieve these goals, LCL comprises four main components: the cell-pair generator, the base encoder, the projection head, and the loss function (Fig. 4). To enable the model to generalize beyond lineage-labeled cells and transfer its learned representations to unlabeled but related datasets, we introduce a complementary unsupervised part alongside the supervised (contrastive) part. This semi-supervised design allows LCL to leverage both labeled cells (i.e., cells with lineage labels) and unlabeled cells (i.e., cells without lineage labels) during training, thereby enhancing robustness and facilitating lineage-label transfer, as we describe below.

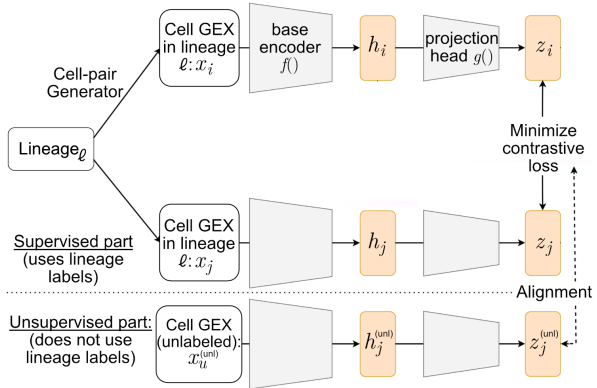


Figure 4. The architecture of LCL uses a contrastive loss on the supervised part, where the lineage labels provide a natural data augmentation mechanism. The cell-pair generator forms positive pairs from the gene expression vectors (GEX) x_i and x_j of two cells with the same lineage label ℓ . The unsupervised part leverages unlabeled cells (i.e., no lineage labels), and the alignment penalty ensures the unlabeled cells’ embeddings are well aligned with the labeled cells’ embeddings.

Cell-pair generator. For the *supervised part*, most contrastive learning frameworks, such as SimCLR (Chen et al., 2020) and SupCon (Khosla et al., 2020), are designed for images, where data augmentation generates positive pairs of images – two images that are numerically distinct but conceptually the same object. In contrast, our method leverages the inherent properties of lineage-barcoded data to define positive pairs – two cells are considered positive pairs if they belong to the same lineage (i.e., $y_i = y_j$ for two cells i and j), while cells from different lineages are treated as negative pairs (i.e., $y_i \neq y_j$). The cell-pair generator first generates all positive pairs used during training. For a specific lineage $L_k \in \{1, \dots, L\}$, its total number of positive pairs is given by $\lceil \alpha \cdot \binom{L_k}{2} \rceil$, where we call $\alpha \in (0, 1]$ the size factor. The cell-pair generator then assigns the positive pairs to different training batches by selecting N cell pairs from N distinct lineages per batch. Within each batch, any

two cells from different lineages are deemed as a negative pair. We then design LCL’s remaining components to maximize the agreement between cells from positive pairs (i.e., cells from the same lineage) and minimize the agreement between negative pairs (i.e., cells from different lineages).

For the *unsupervised part*, we extend this framework to include m unlabeled cells, $\{x_i^{(unl)}\}_{i=1}^m$. In each training iteration, U cells are randomly sampled from the unlabeled dataset. As we will describe, we leverage these unlabeled cells to allow the model to generalize beyond the lineage-barcoded population.

Base encoder. The base encoder $f(\cdot)$ is a neural network that derives an intermediary embedding h_i using x_i , the gene expression vector of each cell i . This is applied to each cell in the pairs we constructed. By default, we use a 3-layer Multilayer Perceptron (MLP) as the base encoder using the Rectified Linear Unit (ReLU) activation function, and batch normalization is applied after each layer to normalize the output. The input and output dimensions of $f(\cdot)$ are p and 64, respectively.

Projection head. The projection head $g(\cdot)$ consists of a two-layer MLP with ReLU that transforms h_n into a lower-dimensional embedding, denoted by $z_n = g(f(x_n))$, where the contrastive loss is applied. In standard contrastive learning frameworks, the projection head is often discarded after training and downstream analyses are performed using the base encoder representation (Gupta et al., 2022). In LCL, both the base-encoder embedding $h_n = f(x_n)$ and the projection-head embedding $z_n = g(f(x_n))$ can be useful for downstream analyses. The projection-head embedding is directly optimized by the contrastive objective and provides a compact lineage-aware representation, while the base-encoder embedding may retain additional biological variation that can benefit some downstream prediction tasks. Therefore, our open-source package outputs both embeddings, and we report both variants in our empirical evaluations when relevant. The output dimension of $g(\cdot)$ is 32.

Loss function. For each batch containing N labeled cell pairs (i.e., $2N$ labeled cells), we define the loss for a positive pair (i, j) using the normalized temperature-scaled cross-entropy loss, a contrastive objective introduced in prior work (Chen et al., 2020),

$$\ell(i, j) = -\log \left(\frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \notin \{i, j\}]} \exp(\text{sim}(z_i, z_k)/\tau)} \right),$$

where $\text{sim}(z_i, z_j)$ denotes the cosine similarity $((z_i)^\top z_j) / (\|z_i\|_2 \|z_j\|_2)$ and $\tau > 0$ is the temperature parameter. The supervised contrastive loss for the labeled cells to achieve Goal #1 is then computed as the

average across all positive pairs,

$$\mathcal{L}_{\text{contrastive}} = \frac{1}{2N} \sum_{(i,j) \in \mathcal{P}} \ell(i,j),$$

where $\mathcal{P}$ is the set of positive (same-lineage) cell pairs. This loss term encourages cells of the same lineage to be close together in the embedding space.

Semi-supervised penalty for unlabeled cells. To learn robust lineage-informative embeddings (Goal #2) and align the embeddings of unlabeled cells with the lineage-informed space learned from labeled cells (Goal #3), we introduce an additional entropy-based penalty term. For each training iteration, U unlabeled cells pass through the same base encoder $f(\cdot)$ and projection head $g(\cdot)$ to obtain embedding vectors $\{z_u^{(\text{unl})}\}_{u=1}^U$. We design the following penalty term to encourage alignment between the labeled and unlabeled embedding spaces. Importantly, this penalty should be interpreted as manifold alignment rather than strict lineage classification. Although unlabeled cells are softly compared to labeled cells in each mini-batch, they are not assigned fixed discrete lineage labels. Instead, the penalty encourages unlabeled cells to align with confident regions of the lineage-informed transcriptomic manifold. Thus, in our biological setting, generalization means mapping unseen lineages onto shared functional cell-state structure, rather than creating a separate orthogonal cluster for every new barcode. Specifically, each unlabeled cell forms a soft assignment to the $2N$ labeled projections using temperature-scaled cosine similarity,

$$p_{u,k} = \frac{\exp(\text{sim}(z_u^{(\text{unl})}, z_k)/\tau)}{\sum_{\ell=1}^{2N} \exp(\text{sim}(z_u^{(\text{unl})}, z_\ell)/\tau)},$$

for $k \in \{1, \dots, N\}$. We then apply an entropy penalty that encourages confident assignments (i.e., low-entropy) between unlabeled cells and nearby labeled neighbors,

$$\mathcal{L}_{\text{penalty}} = \frac{1}{U} \sum_{u=1}^U \left(- \sum_{k=1}^{2N} p_{u,k} \cdot \log p_{u,k} \right).$$

The entropy penalty reduces the embedding gap between labeled and unlabeled cells.

The full training objective combines the supervised contrastive loss and the unsupervised entropy penalty:

$$\mathcal{L}_{\text{train}} = \mathcal{L}_{\text{contrastive}} + \lambda \cdot \mathcal{L}_{\text{penalty}}, \quad (1)$$

where $\lambda > 0$ controls the strength of the semi-supervised regularization. This joint minimization of $\mathcal{L}_{\text{train}}$ improves the model’s robustness on unseen data and allows cells without labels to be mapped to probable lineage neighborhoods in the learned space. As we empirically show, this regularization yields smoother cluster boundaries and lower KNN test error across both labeled and unlabeled datasets.

3.2. Mini-batch construction and semi-supervised sampling

LCL is trained using lineage-based contrastive learning, optionally augmented with a semi-supervised penalty. Training is organized around a cell-pair generator, denoted $\mathcal{C}(\alpha)$, which constructs lineage-based positive cell pairs and assembles them into mini-batches (see Appendix C for details).

For each mini-batch, $\mathcal{C}(\alpha)$ samples N positive cell pairs from distinct lineages whenever possible, yielding $2N$ *labeled cells*. These labeled cells are used to compute the supervised contrastive loss, where positive and negative relationships are defined entirely by lineage membership. In addition, when semi-supervision is enabled, $\mathcal{C}(\alpha)$ independently samples U *unlabeled cells* that do not participate in positive or negative pair construction. Instead, these unlabeled cells contribute only through an entropy-based penalty, which encourages their embeddings to align with the lineage-informed structure learned from the labeled pairs. Unlabeled cells may be drawn either from an external unlabeled dataset or from a held-out unlabeled pool within the same dataset when no external unlabeled data are available. In both cases, unlabeled cells are never used to define supervised contrastive labels and do not affect the construction of positive pairs.

Details on data preprocessing, training, scalability, and diagnostics are provided in Appendix C and Appendix E, where we further document the role of hyperparameters such as the number of positive pairs N , the number of unlabeled cells U per mini-batch, the temperature τ , the size factor α , and the semi-supervised regularization weight λ .

3.3. Downstream application: Identify genes with lineage information via integrated gradients

Lineage barcodes serve as labels that enable LCL to separate heritable, fate-relevant transcriptional programs from dominant but potentially distracting sources of variation (e.g., cell type or time effects). After training, we aim to make the learned lineage-aware embedding interpretable by identifying which genes most strongly drive a cell to reside in its lineage-specific region of the embedding space. Concretely, we treat genes as “lineage-informative” if changing their expression most increases a cell’s alignment with its lineage cluster in the LCL embedding, yielding a ranked gene list that can be inspected for biological plausibility and used for downstream hypothesis generation.

To score gene contributions, we apply Integrated Gradients (IG) to a scalar function that measures how well a cell’s embedding aligns with its lineage centroid (Sundararajan et al., 2017). For each lineage ℓ , we compute its centroid c_ℓ as the mean of the LCL embeddings z_i of cells in that lineage, where $z(x) = g(f(x))$ is the (normalized) projection output

used throughout our downstream analyses. We then define $F_\ell(x) = \cos(z(x), c_\ell)$ and compute IG attributions for each gene j relative to a baseline x' (zero or mean expression): $IG_j(x) = (x_j - x'_j) \int_0^1 \frac{\partial F_\ell(x' + \alpha(x - x'))}{\partial x_j} d\alpha$ approximated numerically. We summarize lineage-level gene importance by averaging $|IG_j|$ across cells within a lineage, and obtain a global ranking by averaging these lineage-level scores across lineages. See Appendix D for more details.

3.4. Downstream application: Predict the future composition of lineages

To quantify cellular fate commitment, we use the LCL embedding of early time point cells to predict the future cell-type composition of their respective lineages. For a system with c terminal states, we represent the fate of lineage i as a c -dimensional proportion vector, where each entry denotes the relative frequency of a specific cell type within that lineage at the final time point. We train a linear decoder that maps individual cell embeddings to logits over terminal cell types, followed by a softmax to obtain a predicted composition distribution.

While standard cross-entropy is well suited for predicting discrete, hard-target classes of individual cells, our goal here is to predict the future composition of an entire lineage. Because each lineage’s fate is represented as a soft target probability distribution over multiple terminal cell types, we optimize and evaluate the linear decoder using KL divergence, which naturally measures the discrepancy between predicted and observed compositional distributions.

This approach allows us to evaluate whether the learned embeddings capture latent, fate-determining signals that drive longitudinal differentiation from cross-sectional data.

4. Experimental Setup

4.1. Datasets

We briefly introduce the datasets in our study; more details are available in Appendix D. In all our analyses, we first perform feature selection to analyze only the top $p = 2000$ highly variable genes, and we log-normalize the gene expression vectors $\{(x_i)\}_{i=1}^n$, as is standard in single-cell literature.

- **Hematopoietic Differentiation** (Weinreb et al., 2020): The authors designed the LARRY protocol to barcode hematopoietic stem cells in mice over a 6-day experiment to study how cells commit to differentiating into certain cell types. We will call this the “LARRY dataset,” which contains 41,093 cells across 11 cell types, with 2,813 lineages, sampled at 3 different time points.
- **Fibroblast Reprogramming** (Biddy et al., 2018): The

authors designed the CellTagging protocol to barcode mouse embryonic fibroblasts when reprogramming them into induced endoderm progenitors (iEPs). Cells in certain lineages failed to reprogram (i.e., “dead-end”). We will call this the “CellTag dataset,” which contains 6,534 cells with 169 different lineages, across 3 main cell types sequenced at various time points in a 28-day experiment (6 time points).

- **Followup Fibroblast Reprogramming** (Jindal et al., 2024): The authors improved the CellTagging protocol to the CellTag-multi protocol and uses this to further study the fibroblast-to-iEP reprogramming system. We will call this dataset the “CellTag-multi dataset,” which contains 22,238 cells across 1,367 lineages, spanning 5 main cell types in a 21-day experiment (3 time points).

4.2. Simulation via pseudo-real data

To evaluate LCL across varying signal-to-noise ratios, we simulate lineages using the LARRY dataset by reassigning labels based on a subset of gene expression values. We define a difficulty parameter $\beta \in \{0.1, 0.3, \dots, 0.9\}$, representing the top fraction of highly variable genes used to induce lineage structure. After ranking genes by variance, we project the top β -fraction onto its principal components and define simulated lineages via Leiden clustering. Lower β values yield benchmarks that are more challenging, with sparse lineage information, ensuring that methods like scVI cannot trivially recover the structure via standard transcriptomic clustering. Five trials are conducted for each β ; see Appendix D for more details.

4.3. Evaluation metrics and baseline methods

After fitting LCL, we evaluate the quality of the learned lineage-aware embedding using various qualitative and quantitative methods.

Evaluation for training cells. To evaluate global lineage coherence, we use the Calinski-Harabasz (CH) Index (Caliński & Harabasz, 1974) to measure cluster separation and cohesion. Additionally, we employ Gene Expression Memory-based Lineage Inference (GEMLI) (Eisele et al., 2024) to assess the predictive power of the latent space. GEMLI trains a random forest classifier on latent cell-pair representations to predict whether two cells originate from the same lineage; we report performance using the Area Under the Precision-Recall Curve (AUPRC) and the Area Under the Receiver Operating Characteristic curve (AUROC). See Appendix C for details.

Evaluation on held-out test cells. We evaluate how well LCL’s embedding generalizes beyond the cells used for training, which is critical for assessing robustness to overfitting and the extent to which the embedding captures biologically meaningful structure. To this end, we split cells into disjoint

training and test sets prior to model fitting. All evaluation metrics are computed exclusively on the held-out test cells, which are not used to train either the LCL encoder or any downstream predictors.

We consider two complementary evaluation metrics. First, we assess whether cells from the same lineage remain close in the embedding space by training a K -nearest neighbor (KNN) classifier on the training-set embeddings and corresponding lineage labels, and evaluating its classification accuracy on the test-set embeddings. Throughout this work, we use $K = 5$. High test accuracy indicates that lineage structure learned during training generalizes to unseen cells.

Second, we evaluate whether the learned embedding preserves information predictive of future cell fate, as described in Section 3.4. Using only the training split, we fit a linear softmax decoder that maps early time-point cell embeddings to lineage-level future cell-type composition vectors. Decoder training and early stopping are performed exclusively on the training split. Performance is then evaluated on the held-out test cells by computing the Kullback–Leibler (KL) divergence between the predicted and observed composition distributions. Lower KL divergence on the test set indicates that LCL successfully isolates latent fate-determining signals that generalize beyond the training data.

Benchmarking baselines and visualization. In the main text, we compare LCL against scVI (Lopez et al., 2018) and Supervised UMAP (McInnes et al., 2018). Additional comparisons with cross-entropy and triplet-loss neural embedding baselines are reported in Appendix E.4. For qualitative assessment, we visualize embeddings using UMAP; for quantitative assessment, we use the KNN lineage-prediction and KL-divergence composition-prediction metrics described above. Implementation details for all baseline methods are provided in Appendix D.8.

5. Results

5.1. LCL reliably extracts the lineage information on pseudo-real data

In our first set of results, we leverage our simulated scLT datasets to confirm LCL’s ability to extract lineage information at varying signal strengths. Using the pseudo-real datasets with varying β , we see that LCL consistently performs well across all difficulty levels (Fig. 5). The high CH Index shows this across all five trials’ β values. This demonstrates LCL’s robustness in extracting lineage information, even when the signal becomes harder to distinguish. In contrast, scVI and Supervised UMAP perform worse at extracting the lineage-dependent signal from the gene expression count matrix at all levels of β .

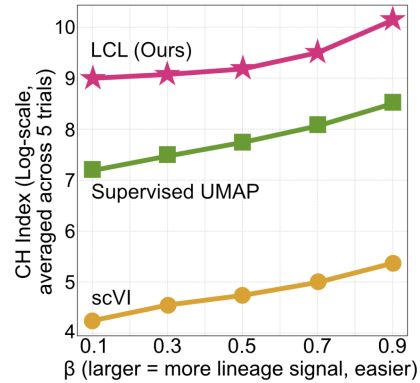


Figure 5. CH Index of different β ’s comparing how LCL compares with scVI and Supervised UMAP, where a higher index means the embedding better separates the different lineages.

5.2. LCL learns lineage-informed embeddings in various biological contexts

After demonstrating LCL’s capabilities on simulated data, we next analyze the LARRY and CellTag datasets for our second analysis. We demonstrate that LCL yields higher-quality embeddings than scVI and Supervised UMAP on both datasets, indicating that cells from the same lineage are more clustered together. Visually, the LCL embeddings form denser clusters in the UMAP, where we observe that the cells from the same lineage form “islands” (Fig. 6B). In contrast, Supervised UMAP shows a visual trend towards lineage clustering but still exhibits significant overlap between cells from different lineages (see Appendix Fig. 15). Meanwhile, scVI embeddings demonstrate a mixture of cells from various lineages without clear separation, likely due to its unsupervised nature (Fig. 6A). These findings are also supported qualitatively by the CH Index, which shows that LCL is more effective at separating cells by lineage and, therefore, better isolates lineage-related information. See Appendix E for more plots and results.

One might question whether the island-like formations in LCL’s UMAP indicate overfitting, so we use the KNN classifier to evaluate the generalizability of LCL’s embeddings.

We train a KNN classifier on the training-cell embeddings to predict their lineages and evaluate the test error on held-out test-cell embeddings.

For the CellTag dataset and the largest 200 lineages from the LARRY dataset, LCL embeddings perform competitively across both held-out lineage prediction and future-composition prediction. In particular, the base-encoder embedding gives the strongest overall KNN and KL-divergence performance, while the projection-head embedding also improves future-composition prediction relative to the baseline embeddings (Table 1).

This further confirms the generalizability of LCL’s embed-

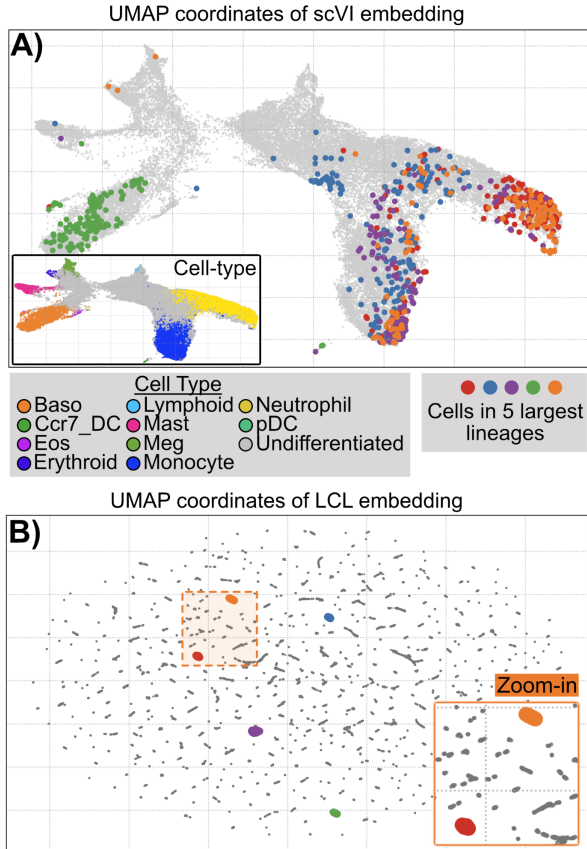


Figure 6. **A)** UMAP of the scVI embedding for the LARRY dataset. The inset shows the cells colored by cell type. **B)** UMAP of the LARRY embeddings. The inset shows a zoom-in of the orange lineage. In both UMAPs, the cells in the five largest lineages are highlighted in different colors in their respective datasets. Surprisingly, although the LCL embedding visually shows “islands,” the embedding performs well on the test set.

dings. More comprehensive results, including comparisons with scVI and Supervised UMAP, other definitions of test cells, and the CH Index, are presented in Appendix E, where LCL consistently achieves higher test accuracy across both datasets. All these results point to somewhat surprising conclusions: although the LCL embedding visually shows “islands,” it performs well on test sets. This suggests that LCL uncovers meaningful biological signals specific to cells within the same lineage, despite their different cell types.

5.3. LCL’s embedding enables improved predictions of future cell type compositions

We next ask whether lineage-informed embeddings improve prediction of future cell fate. For each early-time-point cell, we use its embedding to predict the future cell-type composition of its lineage, as described in Section 3.4. Because each lineage’s fate is represented as a probability distribution over terminal cell types, we evaluate prediction accuracy using KL divergence between the predicted and observed

Table 1. Performance comparison of embedding methods on lineage prediction and future cell type composition prediction. Lower values indicate better performance. The best values are bolded.

Dataset	Method	KNN Test Error ↓	KL Divergence ↓
LARRY (Largest 200)	scVI	0.892	0.549
	Supervised UMAP	0.941	0.903
	LCL (proj)	0.864	0.361
	LCL (base)	0.858	0.335
CellTag	scVI	0.529	0.627
	Supervised UMAP	0.628	0.595
	LCL (proj)	0.459	0.414
	LCL (base)	0.434	0.292

future compositions.

Across both the CellTag dataset and the largest 200 lineages from the LARRY dataset, LCL embeddings achieve lower KL divergence than scVI, and Supervised UMAP baselines (Table 1). The base-encoder embedding performs best in these experiments, suggesting that it retains additional biological variation useful for composition prediction. These findings highlight LCL’s ability to isolate lineage-specific signals that drive cell-fate decisions, enabling it to outperform other methods in predicting future cell-type compositions.

5.4. LCL enables new biological investigations of unlabeled scRNA-seq studies by leveraging lineage-barcoded data

In our final data analysis, we demonstrate that LCL enables the transfer of lineage-informed structure from a lineage-barcoded reference dataset to a related but unlabeled dataset. This setting reflects a common practical scenario in which lineage barcoding is available only for a subset of experiments, while additional datasets measure similar biological systems without explicit lineage annotations. To facilitate this experiment, we use the CellTag and CellTag-multi as the unlabeled and labeled datasets, respectively. Because both datasets capture related biological processes, the learned embedding provides a shared representation space in which lineage-related structure can be transferred. Furthermore, although we treat CellTag as an unlabeled dataset for our experiment, we can use its lineage information to validate LCL’s accuracy in label transfer. See Appendix D for the details of how we perform this analysis.

After fitting LCL on the CellTag-multi (labeled) dataset, we assess how the cells in the CellTag (unlabeled) dataset co-localize based on their withheld lineage labels. After

integrating the datasets together, we assess suitable integration between the two datasets (Fig. 7A). We don’t expect perfect integration, since the lineages in each dataset are distinct. Additionally, when focusing on the CellTag cells, we observe strong lineage co-localization despite LCL not having used their lineage labels during training (Fig. 7B). This demonstrates LCL’s impressive ability to learn the fate-determining biology of lineages even when no lineages are observed.

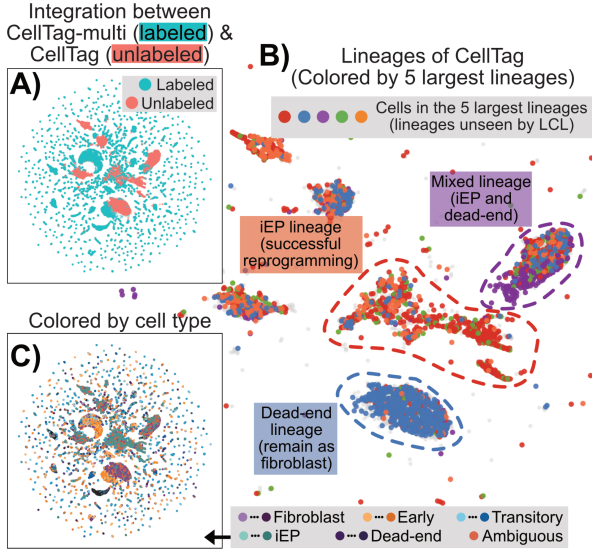


Figure 7. A) LCL embedding’s UMAP showing cells from both datasets to demonstrate the integration. B) Subset of only the cells from the CellTag dataset (unlabeled), colored by the largest 5 lineages. The shown lineages are not used by LCL and are distinct from the lineages in the CellTag-multi dataset (labeled) used during training. Three of the unseen lineages that tightly clustered are highlighted (“islands”), labeled by predominate fate. C) Cells from both datasets colored by cell type.

To evaluate how effectively different embeddings capture lineage-specific signals, we use GEMLI to classify whether two cells are from the same lineage or not. As shown in Table 2, LCL significantly outperforms other methods. Notably, LCL nearly triples the AUPRC of deep generative models (scVI), the log-normalized gene expressions, and a random-guess baseline. These results demonstrate that LCL successfully isolates salient lineage variations from transcriptomic noise, providing a latent representation that is fundamentally more reflective of true fate commitment than standard representation learning approaches.

Table 2. AUPRC and AUROC of GEMLI classification on different embeddings. The best values are bolded.

Metric	LCL	scVI	Log_norm	Random guess
AUPRC	0.600	0.177	0.221	0.027
AUROC	0.825	0.601	0.637	0.5

LCL uniquely uncovers critical biological programs that are statistically invisible to alternative methods (Table 3). We compare against CoSpar (Wang et al., 2022), a framework using sparse optimization to infer state transition maps, and VariancePartition (Hoffman & Schadt, 2016). All three methods model the relation between lineage labels and gene expression. While CoSpar and VariancePartition fail to reach significance, LCL successfully identifies the negative regulation of cell differentiation (adjusted p-value = 0.03), capturing the *Wnt4* and *Sfrp1* axis established in prior work as the primary differentiator between successful iEP conversion and dead-end trajectories. Furthermore, LCL resolves TGF- β signaling (adjusted p-value = 0.04), endothelial cell migration (adjusted p-value = 0.04, featuring *Bmp4*), and extracellular matrix assembly (adjusted p-value = 0.03, featuring *Col1a2*), all of which are hallmark mesenchymal-to-epithelial transition programs described by the original authors (Biddy et al., 2018; Jindal et al., 2024). These results demonstrate LCL’s superior ability to disentangle subtle, fate-determining signals from high-dimensional scRNA-seq data where existing lineage-aware methods fail.

Table 3. Adjusted p-values for pathways of interest across methods. The best values are bolded.

Pathway	LCL	CoSpar	VP
Negative regulation of cell differentiation	0.03	0.22	0.91
Regulation of endothelial cell migration	0.04	0.82	0.74
Regulation of transforming growth factor beta receptor signaling pathway	0.04	0.89	0.81
Extracellular matrix assembly	0.03	0.50	0.85

Lastly, we explore the biological insights LCL uncovers from the CellTag (unlabeled) dataset, despite not being trained on its lineage labels. This is to reinforce the notion that LCL can help advance the biology of fate commitment of *unlabeled* scRNA-seq data, which is much more abundant than lineage-barcoded scRNA-seq data. We perform high-resolution Leiden clustering of the CellTag cells using the LCL embedding to isolate the different “islands.” Then, we identify marker genes distinguishing these clusters using the Wilcoxon test in a one-versus-rest differential expression analysis. Key genes of the failed reprogramming (i.e., *Ptn*, *Sfrp1*) and successful reprogramming into iEP fates (i.e., *Shh*) have both significant p-values and high log fold-changes. We are able to validate the fate-specificity of these genes, since CellTag actually has lineage barcodes. We showcase additional results in Appendix E.

Table 4. Differential expression of LCL clusters. Adjusted p-values, log2 fold-changes, and expressed fate labels are shown.

Gene	Adjusted p-value	log2FC	Expressed fate
<i>Ptn</i>	0.00e+00	3.25	Dead-end
<i>Sfrp1</i>	0.00e+00	3.07	Dead-end
<i>Shh</i>	1.40e-83	3.08	iEP

6. Discussion

We introduce a novel deep learning framework, Lineage-aware Contrastive Learning (LCL), to address challenges in analyzing scLT data. LCL isolates lineage-specific gene expression signals, which are often overshadowed by other biological processes in conventional methods. By leveraging contrastive learning, LCL effectively disentangles lineage-related signals from cell-type- and time-specific signals, enabling us to predict cell fate with higher accuracy and identify critical fate-determining genes.

We demonstrate that contrastive learning is a promising framework for advancing our understanding of fate commitment in lineage-traced cells. Our results show that LCL outperforms existing methods in clustering lineage-specific signals and predicting future cell-type composition. A limitation of our approach is its reliance on sufficient cell counts per lineage to form robust contrastive pairs. While LCL maintains a moderate correlation (Pearson $r = 0.3587$) between lineage size and prediction accuracy, extremely small lineages naturally exhibit higher variance and lower predictive performance, representing a primary failure mode when applying contrastive augmentations to sparse clonal data. This line of future research can have broad implications for improving our understanding of cellular differentiation and development.

Code availability. Our LCL architecture and analysis code are available on GitHub ([SZ-yang/Lineage_aware_ContraLearn](https://github.com/SZ-yang/Lineage_aware_ContraLearn)). Additional details about the code and datasets are in the Appendix S1.

Acknowledgments

We thank Brandon Hadland, the anonymous ICML reviewers, and other members of the Lin Lab for support, helpful discussions, and feedback on this work. Research reported in this manuscript was supported by the National Institute of General Medical Sciences (NIGMS) of the National Institutes of Health (NIH) under award number: R35GM162089.

Impact statement

This paper introduces a machine learning framework designed to improve the interpretation of single-cell lineage tracing data. By isolating subtle, fate-determining signals from complex transcriptomic noise, our work advances the biological understanding of cellular differentiation and development. These advancements have significant potential societal benefits in the field of health and medicine, specifically by facilitating the design of precision therapeutics, improving immunotherapy outcomes, and refining stem cell reprogramming protocols.

References

- Bergen, V., Lange, M., Peidli, S., Wolf, F. A., and Theis, F. J. Generalizing RNA velocity to transient cell states through dynamical modeling. *Nature Biotechnology*, 38 (12):1408–1414, 2020.
- Biddy, B. A., Kong, W., Kamimoto, K., Guo, C., Waye, S. E., Sun, T., and Morris, S. A. Single-cell mapping of lineage and identity in direct reprogramming. *Nature*, 564(7735):219–224, 2018.
- Boyeau, P., Regier, J., Gayoso, A., Jordan, M. I., Lopez, R., and Yosef, N. An empirical Bayes method for differential expression analysis of single cells with deep generative models. *Proceedings of the National Academy of Sciences*, 120(21):e2209124120, 2023.
- Caliński, T. and Harabasz, J. A dendrite method for cluster analysis. *Communications in Statistics - Theory and Methods*, 3(1):1–27, 1974.
- Chen, C., Liao, Y., and Peng, G. Connecting past and present: Single-cell lineage tracing. *Protein & Cell*, 13 (11):790–807, 2022a.
- Chen, T., Kornblith, S., Norouzi, M., and Hinton, G. A simple framework for contrastive learning of visual representations. *International Conference on Machine Learning*, pp. 1597–1607, 2020.
- Chen, Z., King, W. C., Hwang, A., Gerstein, M., and Zhang, J. Deepvelo: Single-cell transcriptomic deep velocity field learning with neural ordinary differential equations. *Science Advances*, 8(48):eabq3745, 2022b.

- Choi, Y., Li, R., and Quon, G. siVAE: Interpretable deep generative models for single-cell transcriptomes. *Genome Biology*, 24(1):29, 2023.
- Chuang, C.-Y., Robinson, J., Lin, Y.-C., Torralba, A., and Jegelka, S. Debiased contrastive learning. In *Advances in Neural Information Processing Systems*, volume 33, pp. 8765–8775, 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/63c3ddcc7b23daa1e42dc41f9a44a873-Abstract.html>.
- Ciortan, M. and Defrance, M. Contrastive self-supervised clustering of scRNA-seq data. *BMC Bioinformatics*, 22(1):280, 2021.
- Eisele, A., Tarbier, M., Dormann, A., Pelechano, V., and Suter, D. Gene-expression memory-based prediction of cell lineages from scRNA-seq datasets. *Nature Communications*, 15(1):2744, 2024.
- Eisenstein, M. The secret life of cells. *Nature Methods*, 17: 7–10, 2020.
- Gupta, K., Ajanthan, T., Hengel, A. v. d., and Gould, S. Understanding and improving the role of projection head in self-supervised learning. *arXiv preprint arXiv:2212.11491*, 2022.
- Guzzetta, A., Koska, M., Rowton, M., Sullivan, K. R., Jacobs-Li, J., Kweon, J., Hidalgo, H., Eckart, H., Hoffmann, A. D., Back, R., et al. Hedgehog–fgf signaling axis patterns anterior mesoderm during gastrulation. *Proceedings of the National Academy of Sciences*, 117(27): 15712–15723, 2020.
- Han, W., Cheng, Y., Chen, J., Zhong, H., Hu, Z., Chen, S., Zong, L., Hong, L., Chan, T.-F., King, I., et al. Self-supervised contrastive learning for integrative single cell RNA-seq data analysis. *Briefings in Bioinformatics*, 23(5):bbac377, 2022.
- Hao, Y., Hao, S., Andersen-Nissen, E., III, W. M. M., Zheng, S., Butler, A., Lee, M. J., Wilk, A. J., Darby, C., Zagar, M., Hoffman, P., Stoeckius, M., Papalexi, E., Mimitou, E. P., Jain, J., Srivastava, A., Stuart, T., Fleming, L. B., Yeung, B., Rogers, A. J., McElrath, J. M., Blish, C. A., Gottardo, R., Smibert, P., and Satija, R. Integrated analysis of multimodal single-cell data. *Cell*, 2021. doi: 10.1016/j.cell.2021.04.048. URL <https://doi.org/10.1016/j.cell.2021.04.048>.
- Hoffman, G. E. and Schadt, E. E. variancePartition: Interpreting drivers of variation in complex gene expression studies. *BMC Bioinformatics*, 17(1):483, 2016.
- Jindal, K., Adil, M. T., Yamaguchi, N., Yang, X., Wang, H. C., Kamimoto, K., Rivera-Gonzalez, G. C., and Morris, S. A. Single-cell lineage capture across genomic modalities with CellTag-multi reveals fate-specific gene regulatory changes. *Nature Biotechnology*, 42(6):946–959, 2024. doi: 10.1038/s41587-023-01931-4.
- Jones, M. G., Yang, D., and Weissman, J. S. New tools for lineage tracing in cancer in vivo. *Annual Review of Cancer Biology*, 7(1):111–129, 2023.
- Khosla, P., Teterwak, P., Wang, C., Sarna, A., Tian, Y., Isola, P., Maschinot, A., Liu, C., and Krishnan, D. Supervised contrastive learning. *Advances in Neural Information Processing Systems*, 33:18661–18673, 2020.
- Lopez, R., Regier, J., Cole, M. B., Jordan, M. I., and Yosef, N. Deep generative modeling for single-cell transcriptomics. *Nature Methods*, 15(12):1053–1058, 2018.
- Mages, S., Moriel, N., Avraham-Davidi, I., Murray, E., Watter, J., Chen, F., Rozenblatt-Rosen, O., Klughammer, J., Regev, A., and Nitzan, M. TACCO unifies annotation transfer and decomposition of cell identities for single-cell and spatial omics. *Nature Biotechnology*, 41(10): 1465–1473, 2023.
- McInnes, L., Healy, J., Saul, N., and Grobberger, L. UMAP: Uniform manifold approximation and projection. *Journal of Open Source Software*, 3(29):861, 2018.
- Nitzan, M. and Brenner, M. P. Revealing lineage-related signals in single-cell gene expression using random matrix theory. *Proceedings of the National Academy of Sciences*, 118(11):e1913931118, 2021.
- Pascual-Ahuir, A., Fita-Torró, J., and Proft, M. Capturing and understanding the dynamics and heterogeneity of gene expression in the living cell. *International Journal of Molecular Sciences*, 21(21):8278, 2020.
- Qin, Q., Bingham, E., La Manno, G., Langenau, D. M., and Pinello, L. Pyro-Velocity: Probabilistic RNA velocity inference from single-cell data. *bioRxiv*, pp. 2022–09, 2022.
- Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., and Sutskever, I. Learning transferable visual models from natural language supervision. *Proceedings of Machine Learning Research*, 139:8748–8763, 2021.
- Schaff, D. L., Fasse, A. J., White, P. E., Vander Velde, R. J., and Shaffer, S. M. Clonal differences underlie variable responses to sequential and prolonged treatment. *Cell Systems*, 15(3):213–226.e9, March 2024. doi: 10.1016/j.cels.2024.01.011.

- Setty, M., Kiseliovas, V., Levine, J., Gayoso, A., Mazutis, L., and Pe’Er, D. Characterization of cell fate probabilities in single-cell data with Palantir. *Nature Biotechnology*, 37(4):451–460, 2019.
- Sha, Y., Qiu, Y., Zhou, P., and Nie, Q. Reconstructing growth and dynamic trajectories from single-cell transcriptomics data. *Nature Machine Intelligence*, 6(1):25–39, 2024.
- Sharma, K., Asp, N. T., Harrison, S., Siller, R., Baumgarten, S. F., Gupta, S., Chollet, M. E., Andersen, E., Sullivan, G. J., and Simonsen, A. Autophagy modulates cell fate decisions during lineage commitment. *Autophagy*, 18(8):1915–1931, 2022.
- Simeonov, K. P., Byrns, C. N., Clark, M. L., Norgard, R. J., Martin, B., Stanger, B. Z., Shendure, J., McKenna, A., and Lengner, C. J. Single-cell lineage tracing of metastatic cancer reveals selection of hybrid EMT states. *Cancer Cell*, 39(8):1150–1162, 2021.
- Spanjaard, B., Hu, B., Mitic, N., Olivares-Chauvet, P., Janjuha, S., Ninov, N., and Junker, J. P. Simultaneous lineage tracing and cell-type identification using CRISPR-Cas9-induced genetic scars. *Nature Biotechnology*, 36(5):469–473, 2018.
- Street, K., Risso, D., Fletcher, R. B., Das, D., Ngai, J., Yosef, N., Purdom, E., and Dudoit, S. Slingshot: Cell lineage and pseudotime inference for single-cell transcriptomics. *BMC Genomics*, 19:1–16, 2018.
- Stuart, T., Butler, A., Hoffman, P., Hafemeister, C., Papalexi, E., III, W. M. M., Hao, Y., Stoeckius, M., Smibert, P., and Satija, R. Comprehensive integration of single-cell data. *Cell*, 177:1888–1902, 2019. doi: 10.1016/j.cell.2019.05.031. URL <https://doi.org/10.1016/j.cell.2019.05.031>.
- Sugiura, A., Andrejeva, G., Voss, K., Heintzman, D. R., Xu, X., Madden, M. Z., Ye, X., Beier, K. L., Chowdhury, N. U., Wolf, M. M., Young, A. C., Greenwood, D. L., Sewell, A. E., Shahi, S. K., Freedman, S. N., Cameron, A. M., Foerch, P., Bourne, T., Garcia-Canaveras, J. C., Karijolic, J., Newcomb, D. C., Mangalam, A. K., Rabinowitz, J. D., and Rathmell, J. C. MTHFD2 is a metabolic checkpoint controlling effector and regulatory T cell fate and function. *Immunity*, 55(1):65–81, 2022.
- Sundararajan, M., Taly, A., and Yan, Q. Axiomatic attribution for deep networks. In *International conference on machine learning*, pp. 3319–3328. PMLR, 2017.
- Trapnell, C., Cacchiarelli, D., Grimsby, J., Pokharel, P., Li, S., Morse, M., Lennon, N. J., Livak, K. J., Mikkelsen, T. S., and Rinn, J. L. The dynamics and regulators of cell fate decisions are revealed by pseudotemporal ordering of single cells. *Nature Biotechnology*, 32(4):381–386, 2014.
- Wagner, D. E. and Klein, A. M. Lineage tracing meets single-cell omics: Opportunities and challenges. *Nature Reviews Genetics*, 21(7):410–427, 2020.
- Wang, S.-W., Herriges, M. J., Hurley, K., Kotton, D. N., and Klein, A. M. CoSpar identifies early cell fate biases from single-cell transcriptomic and lineage information. *Nature Biotechnology*, 40(7):1066–1074, 2022.
- Weinberger, E., Lin, C., and Lee, S.-I. Isolating salient variations of interest in single-cell data with contrastiveVI. *Nature Methods*, 20(9):1336–1345, 2023.
- Weinreb, C., Rodriguez-Fraticelli, A., Camargo, F. D., and Klein, A. M. Lineage tracing on transcriptional landscapes links state to fate during differentiation. *Science*, 367(6479):eaaw3381, 2020. doi: 10.1126/science.aaw3381.
- Wolf, F. A., Angerer, P., and Theis, F. J. SCANPY: Large-scale single-cell gene expression data analysis. *Genome Biology*, 19:1–5, 2018.
- Yang, M., Yang, Y., Xie, C., Ni, M., Liu, J., Yang, H., Mu, F., and Wang, J. Contrastive learning enables rapid mapping to multimodal single-cell atlas of multimillion scale. *Nature Machine Intelligence*, 4(8):696–709, 2022.
- Yu, G., Wang, L.-G., Han, Y., and He, Q.-Y. clusterProfiler: An R package for comparing biological themes among gene clusters. *Omics: A journal of integrative biology*, 16(5):284–287, 2012.

Appendix for "Leveraging Lineage Barcodes as Natural Augmentations for Contrastive Learning of Cell Fate in scRNA-seq Data"

A. Data Availability

The LARRY data (Weinreb et al., 2020) is publicly available and can be downloaded at https://github.com/AllonKleinLab/paper-data/blob/master/Lineage_tracing_on_transcriptional_landscapes_links_state_to_fate_during_differentiation/README.md. The original count data can be additionally downloaded from <https://figshare.com/ndownloader/files/37028569>, (which is linked from <https://github.com/pinellolab/pyrovelocity/> (Qin et al., 2022)). The CellTag data (Biddy et al., 2018) is publicly available and can be downloaded at https://cellrank.readthedocs.io/en/latest/api/_autosummary/datasets/cellrank.datasets.reprogramming_morris.html#cellrank.datasets.reprogramming_morris. The CellTag-multi data (Jindal et al., 2024) is publicly available and can be downloaded at https://figshare.com/projects/CellTag-multi_2023_DataShare/173508, with additional context and files provided with the help of the Morris lab. The melanoma sequential-treatment dataset from Schaff et al. (Schaff et al., 2024) is publicly available through GEO under accession GSE253739 and through the accompanying Figshare project at https://figshare.com/projects/Clonal_differences_underlie_variable_responses_to_sequential_and_prolonged_treatment/192653.

B. Further discussion of contrastive learning literature in relation to LCL

We make some remarks to clarify the scope of our paper relative to the literature. First, our notion of “contrastive” differs from Contrastive Language-Image Pre-Training (CLIP) (Radford et al., 2021) for unsupervised learning, or from methods dedicated to finding biological differences between foreground and background datasets, such as contrastiveVI (Weinberger et al., 2023). Second, we focus on static lineage barcodes here (Weinreb et al., 2020; Biddy et al., 2018), as opposed to mutating lineage barcodes, which capture cell phylogenies that are more complex to model (Spanjaard et al., 2018; Simeonov et al., 2021). Lastly, we note that many methods that learn developmental trajectories from scRNA-seq often use scLT data to validate their method and findings (Qin et al., 2022; Mages et al., 2023; Sha et al., 2024). These existing methods primarily aim to separate cells along the leading axes of variation, often leading to separation by cell type (see Figure 3 in the main text). In contrast, our goal in this paper is to specifically model the lineage barcodes to learn an embedding of single-cell data that enables downstream fate-commitment analyses (see Figure 8).

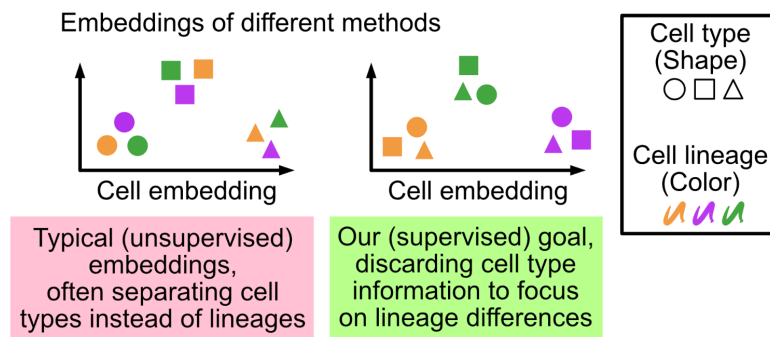


Figure 8. Schematic illustrating the differences between conceptual goal in our paper (right, green) compared to most other papers that model scRNA-seq data (i.e., not leveraging lineage barcodes; left, red).

We note that while contrastive learning has been used to embed scRNA-seq data before (Ciortan & Defrance, 2021; Han et al., 2022; Yang et al., 2022), we are, to our knowledge, the first to explore this framework on scLT data, treating the lineage barcodes as data augmentations.

C. Details of LCL

In Section C.1, we describe more details about the training, the formation of the test set, and the hyperparameters that dictate LCL’s performance. In Section C.2, we document how the cell-pair generator (i.e., constructing positive pairs from

lineage labels and including unlabeled cells) works. In Section C.3, we provide the pseudocode for LCL. In Section C.4, we document how the training and testing cells are partitioned, when there are both labeled and unlabeled cells involved during training.

C.1. Details about training, architecture, and hyperparameters.

Training. The dataset is split into training, validation, and test sets. The validation set is used for early stopping to prevent overfitting – if the normalized temperature-scaled cross-entropy loss (NT-Xent) loss on the validation set stops improving after a set number of epochs, training is halted. We optimize the base encoder and projection head neural networks using the AdamW optimizer.

Forming Test Set. We create two types of test sets to evaluate different aspects of LCL’s generalization ability. The first test set includes 10% of the cells from each lineage, meaning it contains lineages seen during training. The second, more challenging set withholds 10% of the entire lineages, testing the model’s ability to generalize to unseen lineages and its potential applicability to datasets that do not have lineage barcodes.

For test sets constructed in the former method, this set is used in the evaluations described in the “Evaluation when unlabeled cells with shared lineages with the labeled cells” section. Specifically, as illustrated in Figure 9, while the training cells are used to fit LCL’s embedding and downstream classifiers, the test cells are used to evaluate the accuracy of the classifiers.

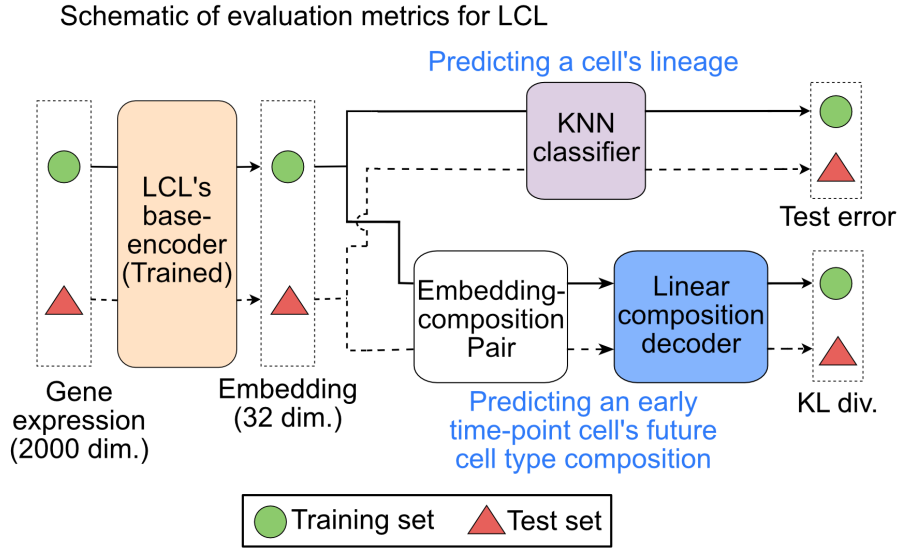


Figure 9. Schematic illustrating two evaluation metrics we apply to the LCL embedding. One is for predicting a cell’s lineage (top), and another is for predicting an early time-point cell’s future cell type composition (bottom).

Choosing Hyperparameters. The LCL model has three primary hyperparameters: the size factor α , the batch size N , and the temperature τ . Size factor $\alpha \in (0, 1]$ and batch size N are both used in the cell-pair generator step. A larger α would yield more pairwise comparisons within and between lineages during training, leading to a better-generalizing embedding at the expense of a higher computational burden. A larger batch size $N \in \mathbb{Z}_+$ means more negative examples are in each batch, facilitating faster convergence and improving performance, especially with fewer training epochs. We use $\alpha = 0.3$ and $N = 100$ as default, but vary them depending on computing resources. The temperature $\tau > 0$ in the NT-Xent loss function controls how the model weights the negative pairs. A small temperature sharpens similarity scores, focusing on harder negatives, which can enhance learning but risks instability or overfitting. In contrast, a larger temperature smoothens the distribution, reducing the emphasis on harder negatives, which may slow learning or yield less distinct embeddings. We find that $\tau = 0.5$ is an appropriate choice for analyzing scLT data.

C.2. Details on cell-pair generator $\mathcal{C}(\cdot)$.

We denote the cell-pair generator as $\mathcal{C}(\alpha)$, where α is the size factor. The role of $\mathcal{C}(\alpha)$ is to construct all lineage-based positive cell pairs used by LCL and to organize them into mini-batches prior to training, while additionally providing unlabeled cells for the semi-supervised penalty. See Figure 10 of a schematic of positive and negative pairs.

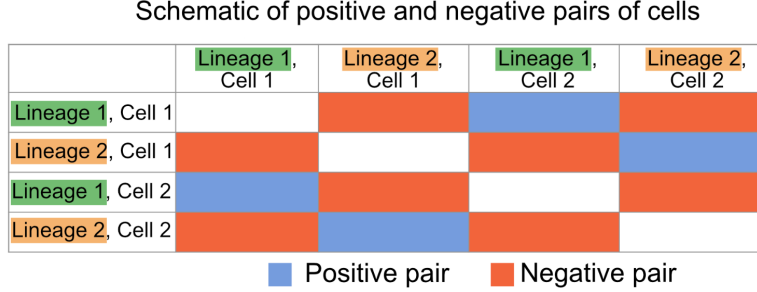


Figure 10. Schematic of positive and negative pairs based on 4 cells split across 2 lineages.

For each lineage L_i , $\mathcal{C}(\alpha)$ generates $\lceil \alpha \cdot \binom{|L_i|}{2} \rceil$ positive cell pairs. If $\lceil \alpha \cdot \binom{|L_i|}{2} \rceil < |L_i|/2$, we instead generate $\lceil |L_i|/2 \rceil$ pairs to ensure sufficient coverage. We further enforce that every cell appears in at least one positive pair across all generated pairs. Let S denote the total number of positive pairs generated across all lineages.

These S pairs are partitioned into $\lceil S/N \rceil$ mini-batches. Each mini-batch selects N positive pairs from N distinct lineages without replacement whenever possible. If fewer than N lineages remain, the final mini-batch is completed by sampling additional pairs from previously used lineages. The $2N$ cells formed by these N pairs constitute the labeled portion of the batch and are used to compute the supervised contrastive loss.

In addition, for each mini-batch, $\mathcal{C}(\alpha)$ independently samples U cells from an unlabeled dataset (or from held-out cells in the same dataset when no external unlabeled data are provided). These unlabeled cells are not used to form positive or negative pairs. Instead, they are incorporated into the training process through an auxiliary entropy-based penalty that encourages their embeddings to align with the lineage-informed structure learned from the labeled pairs.

C.3. Main Algorithm.

After the cell-pair generator has created the batches, they are passed through the main algorithm, and its pseudocode for this process is provided in Algorithm 1.

C.4. Details of train/test cells and labeled/unlabeled cells

Train/validation/test cells (for evaluating generalization). We first split cells into *training*, *validation*, and *test* sets. The validation set is used only for early stopping during LCL training. After training, we compute embeddings for both training and test cells by running the trained encoder forward on each set (without using test labels to fit either the encoder or downstream predictors). In the “train-test split” setting, the test set contains (approximately) 10% of cells from each lineage, so the set of lineages overlaps between train and test; in the “unseen-lineages” setting, entire lineages are withheld so train/test lineages are disjoint.

Labeled vs. unlabeled cells (within an LCL mini-batch). Within each LCL mini-batch, *labeled cells* refer to the $2N$ cells that participate in the lineage-based positive pairs (and hence determine positive vs. negative relationships for the contrastive loss). Separately, when semi-supervision is enabled, we additionally sample U *unlabeled cells* that do not form positive pairs and are not treated as negatives; instead, they contribute only through the auxiliary entropy-based penalty term. In our implementation, unlabeled cells are drawn either from (a) an external unlabeled dataset or (b) a held-out unlabeled pool when no external dataset is provided. In all cases, unlabeled cells are *not* used to define the supervised contrastive labels (i.e., they never create positive pairs).

Interaction with KNN-based lineage prediction. For the KNN-based evaluation, the distinction between training and test cells is the only split that directly enters the analysis. Specifically, we train a KNN classifier using the embeddings of

Algorithm 1 Lineage-aware Contrastive Learning (LCL) with optional semi-supervision

```

0: Input: Batch lineage count  $N \in \mathbb{Z}_+$ , size factor  $\alpha \in (0, 1]$ , temperature  $\tau > 0$ , penalty weight  $\lambda \geq 0$ , unlabeled batch
   size  $U \in \mathbb{Z}_{\geq 0}$ , cell-pair generator  $\mathcal{C}(\alpha)$ 
0: Initialize base encoder  $f(\cdot)$  and projection head  $g(\cdot)$ 
0: for each batch of  $N$  lineage pairs from  $\mathcal{C}(\alpha)$  do
0:   for  $i \in \{1, \dots, N\}$  do
0:     Sample one positive pair  $(x_{2i-1}, x_{2i})$  from a lineage  $L_i$ 
0:      $z_{2i-1} \leftarrow \text{norm}(g(f(x_{2i-1})))$ .
0:      $z_{2i} \leftarrow \text{norm}(g(f(x_{2i})))$ 
0:   end for
0:    $Z \leftarrow [z_1, \dots, z_{2N}]$ 
0:   for Each anchor cell  $m \in \{1, \dots, 2N\}$  do
0:      $p(m) \leftarrow$  the index of  $m$ 's positive partner in  $Z$ 
0:      $\ell_m \leftarrow -\log \frac{\exp(\text{sim}(z_m, z_{p(m)})/\tau)}{\sum_{k=1}^{2N} \mathbb{1}[k \neq m] \exp(\text{sim}(z_m, z_k)/\tau)}$ 
0:   end for
0:    $\mathcal{L}_{\text{con}} \leftarrow \frac{1}{2N} \sum_{m=1}^{2N} \ell_m$ 
0:   if  $\lambda > 0$  and  $U > 0$  then
0:     Sample unlabeled cells  $\{x_u^{(\text{unl})}\}_{u=1}^U$ 
0:      $u_u \leftarrow \text{norm}(g(f(x_u^{(\text{unl})})))$  for  $u \in \{1, \dots, U\}$ 
0:      $p_{u,k} \leftarrow \text{softmax}_k(\text{sim}(u_u, z_k)/\tau)$  for  $k \in \{1, \dots, 2N\}$ 
0:      $\mathcal{L}_{\text{pen}} \leftarrow \frac{1}{U} \sum_{u=1}^U \left( -\sum_{k=1}^{2N} p_{u,k} \log p_{u,k} \right)$ 
0:      $\mathcal{L} \leftarrow \mathcal{L}_{\text{con}} + \lambda \cdot \mathcal{L}_{\text{pen}}$ 
0:   else
0:      $\mathcal{L} \leftarrow \mathcal{L}_{\text{con}}$ 
0:   end if
0:   Update  $f(\cdot), g(\cdot)$  by minimizing  $\mathcal{L}$ 
0: end for
0: return trained  $f(\cdot)$  and  $g(\cdot)$  (embedding  $z = \text{norm}(g(f(x))) = 0$ )

```

training-set cells together with their lineage labels, and we then evaluate classification accuracy on the embeddings of the held-out test cells. The labeled/unlabeled distinction applies only during LCL training and influences the learned embedding, but it does not affect how the KNN classifier is trained or evaluated, which is strictly based on the train/test partition.

Interaction with KL-divergence-based fate composition prediction. For the KL-divergence evaluation, we again use a train/test split to assess generalization, ensuring that prediction targets are constructed separately within each split to avoid information leakage across time points. For a given split, lineage-level fate compositions are computed using only future-time-point cells from that split (e.g., Day 28 in the CellTag analysis) and are then assigned as targets to early-time-point cells from the same split (e.g., Day 12 in the CellTag analysis). Then, the linear decoder is trained using early-time-point embeddings from the training set with composition targets derived exclusively from training-set future cells, and the KL divergence is evaluated on the test set using targets derived exclusively from test-set future cells. Notably, we fit the linear decoder on only the training split. The linear decoder is trained with AdamW (lr 5×10^{-3} , weight decay 10^{-4} , batch size 256, up to 5000 epochs) to minimize a KL-divergence loss (`KLDivLoss`, `batchmean`, on log-softmax logits), with early stopping (patience 150) on a 20% validation split.

D. Data analysis details

In Section D.1, we document the overall preprocessing pipeline prior to using LCL. In Sections D.2, D.3, D.4, and D.5, we provide summary statistics of the LARRY, CellTag, CellTag-multi, and melanoma sequential-treatment datasets, respectively, and also document the hyperparameters we use for each dataset's analyses. In Section D.6, we document how we simulated pseudo-real data. In Sections D.7 and D.9, we provide more details on the CH Index and compositional analysis, respectively. In Section D.10, we document the full procedure on how we computed the integrated gradients. In Section D.11, we provide

the details of how we apply other (existing) methods to investigate fate dynamics. In Section D.12, we document how we perform the batch correction when analyzing the CellTag-multi and CellTag dataset together.

D.1. General workflow.

We use Scanpy (Wolf et al., 2018) for preprocessing the input data for the LCL model. Initially, we remove cells that lack a lineage barcode and cells that belong to lineages with fewer than five cells. Afterward, we log-normalize the counts and select the top 2000 highly variable genes (HVGs) for further analysis.

D.2. Additional details about the LARRY dataset

Biological insights. The authors used isolated unipotent (i.e., one future fate) and pluripotent (i.e., multiple future fates) progenitor cells, using single-cell RNA sequencing to track their fate progression over six days (Weinreb et al., 2020). The primary objective is to determine whether HSPCs follow a continuum of states or discrete hierarchical paths during differentiation. Their findings revealed that progenitors form a structured continuum, and specific progenitors belonged to lineages that resulted in cells differentiating into either a single cell type at a later time (i.e., unipotent) or multiple cell types (i.e., pluripotent).

Table 5 and 6 show the summary of cell types and time points. Specifically, the abbreviations of cell types are basophils (“Baso”), mast cells (“Mast”), megakaryocytes (“Meg”), eosinophils (“Eos”), migratory dendritic cells with marker Ccr7+ (“Ccr7_DC”), and plasmacytoid dendritic cells (“pDC”). Figure 11B shows the number of cells per lineage.

Table 5. Cell Type Distribution in the LARRY Dataset

Cell Type	Count	Proportion
Undifferentiated	18426	0.448
Neutrophil	7555	0.184
Monocyte	7350	0.179
Baso	5075	0.124
Mast	1249	0.030
Meg	828	0.020
Erythroid	315	0.008
Eos	147	0.004
Lymphoid	78	0.002
Ccr7_DC	39	0.001
pDC	31	0.001

Table 6. Time Point Distribution in the LARRY Dataset

Time Point	Count	Proportion
Day 6	27081	0.66
Day 4	12266	0.30
Day 2	1746	0.04

Training details. The hyperparameters we use to train the LCL model with the full LARRY dataset (as shown in Figure 6 in the main text) are $\{N = 270, \alpha = 0.4, \tau = 0.5\}$. When using the LARRY dataset to evaluate lineage predictions, the test set includes 10% of cells from each lineage (which we call the “train-test split”), and we use $\{N = 270, \alpha = 0.5, \tau = 0.5\}$. Additionally, when using the LARRY dataset to evaluate the prediction of lineages where the test set includes all cells from 10% of the lineages (which we call the “unseen lineages split”), we use $\{N = 250, \alpha = 0.45, \tau = 0.5\}$.

During our analysis of the LARRY dataset, we observed many lineages with very few cells (see Figure 11B). We observe that these lineages with few cells often deteriorated LCL’s performance because there are not enough cell pairs for our contrastive loss function. Hence, we also experimented with two other variants of the LARRY dataset, in which we subset the cells to only the largest lineages. For example, when training only on the cells in the 200 largest lineages from the

LARRY dataset with a train-test split, the hyperparameters are $\{N = 150, \alpha = 0.8, \tau = 0.5\}$, while for the unseen lineages split, we use $\{N = 140, \alpha = 0.8, \tau = 0.5\}$. Table 16 reports the number of cells in each subset of the LARRY dataset.

We change the batch size N when analyzing different cell subsets in the LARRY data because the number of available lineages cannot be smaller than N . The differences in batch size N and size factor α for the same subset of cells under different experimental conditions are due to system RAM limitations. For scVI and Supervised UMAP, we use the default settings.

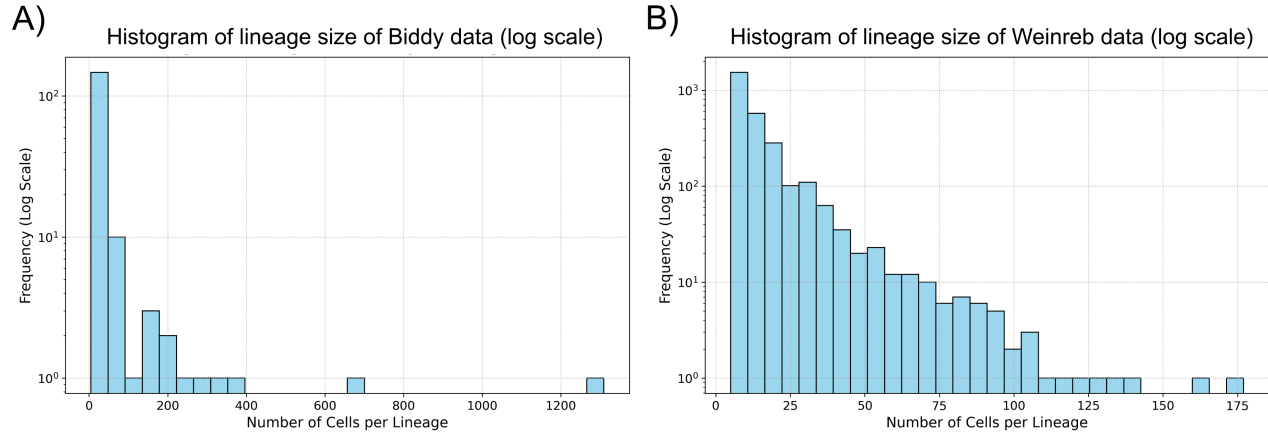


Figure 11. A) Number of cells per lineage in the CellTag dataset. B) Number of cells per lineage in the LARRY dataset.

D.3. Additional details about the CellTag dataset.

Biological insights. CellTagging, a sequential round of lentiviral barcoding followed by single-cell RNA sequencing, allows the authors to track both the lineage history and identity of individual cells throughout the 28-day reprogramming process (Biddy et al., 2018). See Figure 12 for a visualization of the dataset. The authors aimed to understand cellular heterogeneity and the distinct lineage trajectories that emerge during direct reprogramming, and to identify molecular markers associated with successful lineage conversion. The study revealed two main trajectories: one leading to successful reprogramming into iEPs and another resulting in a “dead-end” state, with early gene expression patterns of specific genes predicting which path cells would take.

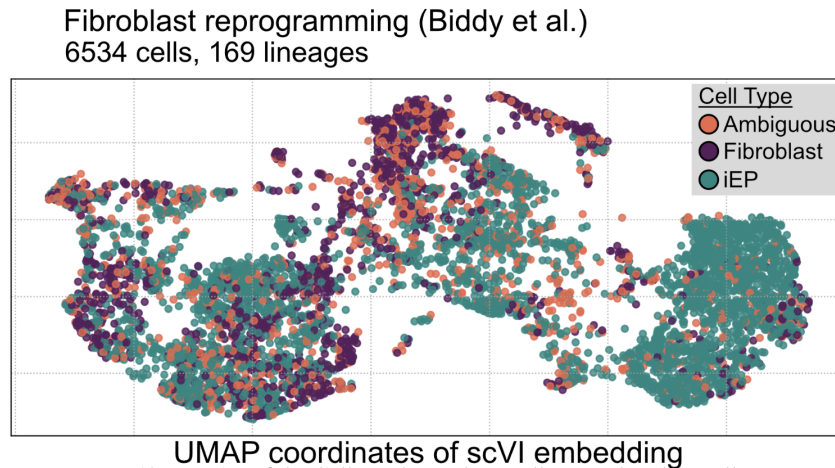


Figure 12. UMAP of the CellTag data, where cells are colored by cell type.

Table 7 and 8 summarize the cell types and time points. Figure 11A shows the number of cells per lineage.

Table 7. Cell Type Distribution in the CellTag Dataset

Cell Type	Count	Proportion
iEP	3669	0.56
Fibroblast	1508	0.23
Ambiguous	1357	0.21

Table 8. Time Point Distribution in the CellTag Dataset

Time Point	Count	Proportion
Day 28	2451	0.375
Day 21	1940	0.297
Day 15	942	0.144
Day 12	967	0.148
Day 9	213	0.033
Day 6	21	0.003

Training details. The hyperparameters we use to train the LCL model with the CellTag dataset are $\{N = 50, \alpha = 0.04, \tau = 0.5\}$ in all experiments. For scVI and Supervised UMAP, we use the default settings.

D.4. Additional details about the CellTag-multi dataset.

Biological insights. CellTag-multi extends CellTagging by enabling recovery of heritable lentiviral CellTag barcodes in both scRNA-seq and scATAC-seq, allowing clonal tracking of transcriptional and epigenomic states within the same biological system (Jindal et al., 2024). In the fibroblast-to-iEP direct reprogramming setting, the authors used this multi-omic lineage readout to further link the early cellular state to the eventual reprogramming outcome, beyond what is previously reported in the literature (Biddy et al., 2018). This dissects fate-specific gene-regulatory programs that distinguish on-target iEP trajectories from off-target/dead-end fates.

Tables 9 and 10 summarize the cell types and time points. Here, ‘Transition’ denotes states in between MEFs and reprogrammed/dead-end identities, while ‘Early’ denotes early transition reprogramming/transition state shortly after induction. We note that the original data provided additional subtypes for each of these major cell types, which is why Figure 7C in the main text shows different shades for each cell type. We also note that while there are explicitly labeled “Dead-end” cells in the CellTag-multi dataset, these correspond to “Fibroblast” cells in the CellTag dataset that are sequenced at later time points (i.e., dead-end, in the sense that the cells failed to reprogram into iEPs).

Table 9. Major Cell Type Distribution in CellTagMulti Dataset

Major Cell Type	Count	Proportion
Transition	9596	0.432
Early	7330	0.330
Fibroblast	2446	0.110
iEP	1761	0.079
Dead-end	1105	0.050

Table 10. Time Point Distribution in the CellTag-multi Dataset

Time Point	Count	Proportion
Day 3	10890	0.49
Day 12	3258	0.15
Day 21	8090	0.36

Training details. The specific details of our analysis integrated CellTag-multi and CellTag are located in Appendix E.6.

D.5. Additional details about the melanoma sequential-treatment dataset.

Biological setting. To evaluate LCL in a biological system distinct from hematopoietic differentiation and fibroblast reprogramming, we analyzed a melanoma clonal-tracing dataset from Schaff et al. (Schaff et al., 2024). This study combined single-cell RNA sequencing with lineage barcoding to track melanoma cell clones through prolonged and sequential drug treatments. Cells are first exposed to one treatment condition, including cisplatin (Cis), cobalt chloride (CoCl₂), or dabrafenib plus trametinib (Dab/Tram), and surviving clones are then subjected to a second-stage treatment. Individual clones exhibited heterogeneous downstream outcomes, including growth, survival, or death, under different treatment sequences. This dataset therefore provides a complementary setting for assessing LCL in a cancer system shaped by treatment-induced selective pressure rather than developmental differentiation.

Analysis setup. We apply LCL to cells sampled after the first-stage treatment, using clone identities as lineage labels during contrastive training. We evaluate the learned embedding using two downstream tasks: KNN lineage prediction against a random baseline, and KL-divergence prediction of future treatment-response composition against a naive baseline. For the compositional analysis, we defined each lineage’s future outcome by the composition of its surviving descendants across second-stage treatment conditions. This yields a treatment-response composition vector for each clone. We then train a linear decoder on first-stage LCL embeddings to predict these clone-level future survivor compositions and evaluate its performance using KL divergence. The quantitative results are reported in Appendix E.5.

D.6. Simulation details on pseudo-real data.

We report additional details on how we simulated the pseudo-real scLT dataset. To generate simulated lineages, genes are first ranked by variance across all cells. For a specific difficulty level β , we select the top $\beta \cdot p$ genes and reduce the dimensionality to $k = \max\{50, \lceil \beta \cdot p \rceil\}$ principal components. Leiden clustering with default parameters is then applied to the resulting k PCs to assign ground-truth lineage labels.

We report the Leiden clustering resolution for various difficulty levels β in Table 11. Recall that a larger β represents a dataset that is “easier,” whereby there is more information to predict a cell’s lineage from its gene expression profile. We change the cluster resolution across different values of β to accentuate the differences across β better. We also perform a separate experiment, setting the cluster resolution to 45 for all values of β . In that experiment, we see the same trend as in Figure 5 in the main text – the CH Index is magnitudes higher with LCL than with Supervised UMAP, which is magnitudes higher than that with scVI. However, because the differences between the two methods (with β fixed) are much larger than the differences between β ’s (for the same method), we choose to vary the cluster resolution instead to accentuate both the differences across methods and β ’s at the same time.

Table 11. Parameters for generating the pseudo-real datasets

β	Clustering res.	Num. of lineages
0.1	65	1221
0.3	55	970
0.5	45	797
0.7	35	597
0.9	25	385

D.7. Additional details about the Calinski-Harabasz Index.

Suppose there are n cells assigned to K clusters. The Calinski-Harabasz (CH) Index evaluates clustering by comparing the between-cluster dispersion (BCSS) to the within-cluster dispersion (WCSS), adjusted for the number of clusters and data points,

$$\text{CH Index} = \frac{\text{BCSS}/(K - 1)}{\text{WCSS}/(n - K)}.$$

In essence, a higher CH Index indicates better cluster separation and compactness.

Here, the Between-Cluster Sum of Squares (BCSS) captures the separation between clusters by measuring the squared

distances between each cluster centroid and the overall data centroid,

$$\text{BCSS} = \sum_{k=1}^K n_k \|c_k - c\|_2^2,$$

where n_k denotes the number of cells in cluster k , c_k is cluster center for cluster k , and c is the global center of the dataset. The Within-Cluster Sum of Squares (WCSS) measures how compact the clusters are by summing the squared distances between points and their cluster centroids,

$$\text{WCSS} = \sum_{k=1}^K \sum_{i \in C_k} \|x_i - c_k\|_2^2,$$

where $C_k \subseteq \{1, \dots, n\}$ is the set of indices in the dataset of n cells that are assigned to cluster k , and the i th cell has coordinates x_i .

D.8. Additional details about baseline embedding methods used in benchmarking.

To benchmark LCL, we compare its learned embedding against four alternative representations: scVI, Supervised UMAP, a cross-entropy-trained neural embedding, and a triplet-loss-trained neural embedding. For all methods, we use the resulting low-dimensional representations as inputs to the same downstream evaluation pipelines described above: (i) K -nearest-neighbor lineage prediction, and (ii) future cell-type composition prediction using a linear decoder followed by KL-divergence evaluation. Thus, differences in downstream performance reflect the quality of the learned embeddings rather than differences in the evaluation procedure.

scVI. We train scVI (Lopez et al., 2018) on the training split using raw count data as input, following the standard `scvi-tools` workflow. For each dataset, counts are stored in a dedicated `counts` layer, and scVI is trained using the SCVI model with its default optimization procedure. To ensure consistent feature spaces between training and test cells, highly variable genes are selected using only the training data, and the test data are subset to the same gene set before extracting latent representations. For the benchmarking results reported in the main text, we use 10- and 64-dimensional scVI latent embeddings. After fitting scVI on the training data, we extract latent representations for both training and held-out test cells using `model.get_latent_representation()`.

Supervised UMAP. We use Supervised UMAP (McInnes et al., 2018) as a nonparametric supervised embedding baseline. The model is fit on the training data using lineage labels as supervision and then applied to the held-out test cells to obtain out-of-sample embeddings. We use a 10-dimensional embedding space, i.e., `n_components = 10`. These embeddings are then passed into the same KNN and KL-divergence evaluation workflows used for LCL.

Cross-entropy-trained neural embedding. We also benchmarked against a standard lineage-classification network trained with cross-entropy loss. This baseline tests whether directly predicting lineage identities yields an embedding that is useful for downstream fate-related analyses.

The model consists of a multilayer perceptron encoder followed by a linear classification head. Given an input gene-expression vector $x_i \in \mathbb{R}^p$, the encoder maps x_i to a 32-dimensional embedding $h_i \in \mathbb{R}^{32}$:

$$h_i = \phi_{\text{CE}}(x_i).$$

The encoder architecture is

$$p \rightarrow 1024 \rightarrow 256 \rightarrow 32,$$

with batch normalization and ReLU activation after each hidden layer and after the final embedding layer. A linear classifier then maps the embedding to logits over the set of lineage labels:

$$o_i = Wh_i + b,$$

where $o_i \in \mathbb{R}^C$ and C denotes the number of lineage classes present in the training split.

The model is trained using the standard multiclass cross-entropy objective,

$$\mathcal{L}_{\text{CE}} = -\frac{1}{n} \sum_{i=1}^n \log \left(\frac{\exp(o_{i,y_i})}{\sum_{c=1}^C \exp(o_{i,c})} \right),$$

where y_i is the lineage label for cell i . Only training-set lineage labels are used to define the classification targets.

We train this model for 50 epochs using AdamW with learning rate 3×10^{-4} , weight decay 10^{-6} , and mini-batch size 256. After training, we discard the final classification layer and use the 32-dimensional hidden representation h_i as the embedding for both training and test cells. These embeddings are evaluated using the same KNN and KL-divergence procedures used for LCL.

Triplet-loss neural embedding. We further compare LCL with a metric-learning baseline trained using triplet margin loss. This baseline directly encourages cells from the same lineage to lie closer together in embedding space than cells from different lineages, but unlike LCL, it relies on sampled anchor-positive-negative triplets rather than LCL’s batchwise contrastive objective.

For each anchor cell x_a with lineage label y_a , we randomly sampled: (i) a positive cell x_p from the same lineage, satisfying $y_p = y_a$, and (ii) a negative cell x_n from a different lineage, satisfying $y_n \neq y_a$. The model encoder maps each input to a 32-dimensional embedding,

$$z_a = \phi_{\text{tri}}(x_a), \quad z_p = \phi_{\text{tri}}(x_p), \quad z_n = \phi_{\text{tri}}(x_n),$$

which is then ℓ_2 -normalized before computing the loss.

The encoder architecture is

$$p \rightarrow 1024 \rightarrow 256 \rightarrow 32,$$

with batch normalization and ReLU activation after the first two hidden layers. Unlike the cross-entropy baseline, no classification head is used. The final embedding is normalized as

$$\tilde{z} = \frac{z}{\|z\|_2}.$$

We optimize the standard triplet margin loss,

$$\mathcal{L}_{\text{triplet}} = \frac{1}{B} \sum_{i=1}^B \max \{d(\tilde{z}_{a_i}, \tilde{z}_{p_i}) - d(\tilde{z}_{a_i}, \tilde{z}_{n_i}) + m, 0\},$$

where B is the mini-batch size, $d(\cdot, \cdot)$ is the Euclidean distance, and the margin is set to $m = 1.0$.

We train the triplet-loss model for 50 epochs using AdamW with learning rate 3×10^{-4} , weight decay 10^{-6} , and mini-batch size 256. After training, the 32-dimensional normalized embeddings are extracted for both training and held-out test cells and evaluated using the same KNN and KL-divergence benchmarks as the other methods.

D.9. Additional details about the compositional analysis.

In the LARRY dataset, we designate Day 2 as the early time point, with all cells labeled Undifferentiated, and Day 6 as the future time point. We focus on only the Day 2 cells in the Monocyte-Neutrophil trajectory, as defined by the original authors (Weinreb et al., 2020). By Day 6, the cell types diversify, and the three largest cell type populations are neutrophils, monocytes, and undifferentiated cells. We compute our compositional vector among these three cell types on Day 6. We further retain only lineages with at least 5 cells at the future time point (Day 6), so that each lineage’s composition target is estimated from a sufficient number of future cells.

In the CellTag dataset, we designate Day 12 as the early time point and Day 28 as the future time point. Both time points include three cell types: iEP, fibroblast, and ambiguous. However, by Day 28, the iEP population are notably larger proportionally, making up 72% of the total cells, compared to 53% at Day 12. We compute our compositional vector among these three cell types on Day 28. Likewise, we retain only lineages with at least 3 cells at the future time point (Day 28).

D.10. Additional details about our usage of integrated gradients.

To identify genes that most strongly contribute to lineage-resolved structure learned by LCL, we apply Integrated Gradients (IG) (Sundararajan et al., 2017) to a trained LCL model in a post hoc manner. Rather than attributing individual embedding coordinates, we define a *lineage-specific scalar objective* measuring how strongly a cell aligns with its lineage centroid in the learned embedding space.

Embedding and lineage centroids. Let $x \in \mathbb{R}^p$ denote the (log-normalized) gene-expression vector for a cell across p genes. LCL consists of a base encoder $f(\cdot)$ and projection head $g(\cdot)$, producing a normalized embedding

$$z(x) = \text{norm}(g(f(x))) \in \mathbb{R}^d, \quad \text{norm}(u) = \frac{u}{\|u\|_2 + \varepsilon},$$

with a small $\varepsilon > 0$ for numerical stability. For each lineage ℓ , let $\mathcal{I}_\ell$ be the set of indices of cells in that lineage. We compute the lineage centroid in embedding space as,

$$\mu_\ell = \text{norm}\left(\frac{1}{|\mathcal{I}_\ell|} \sum_{i \in \mathcal{I}_\ell} z(x_i)\right).$$

In practice, embeddings $z(x_i)$ are computed for all cells without gradients; centroids are computed for the subset of lineage considered in the IG analysis.

Lineage-specific scalar attribution target. For a fixed lineage ℓ , we define a scalar function

$$F_\ell(x) = \cos(z(x), \mu_\ell).$$

Intuitively, $F_\ell(x)$ measures how aligned a cell’s embedding is with the lineage’s centroid direction.

Integrated Gradients. Given a baseline input x' (we use $x' = 0 \in \mathbb{R}^p$, i.e. all genes set to zero; alternatively one may use the global mean expression), the IG attribution for gene $j \in \{1, \dots, p\}$ is

$$\text{IG}_j(x; x', F_\ell) = (x_j - x'_j) \int_0^1 \frac{\partial F_\ell(x' + \alpha(x - x'))}{\partial x_j} d\alpha.$$

We approximate the integral using a fixed number of steps (default: $m = 100$) with Gauss–Legendre quadrature (as implemented by the Python package `Captum`).

Lineage-level and global gene scores. For each lineage ℓ , we randomly sample up to $M = 200$ cells (or all cells if $|\mathcal{I}_\ell| < M$) and compute IG for each sampled cell. We summarize per-lineage gene importance by the mean absolute attribution:

$$s_{\ell,j} = \frac{1}{|S_\ell|} \sum_{i \in S_\ell} |\text{IG}_j(x_i; x', F_\ell)|,$$

where $S_\ell \subseteq \mathcal{I}_\ell$ is the set of sampled cell indices for lineage ℓ . To obtain a global ranking across lineages, we average across the L analyzed lineages:

$$s_j = \frac{1}{L} \sum_{\ell=1}^L s_{\ell,j}.$$

Genes with large s_j are those whose expression consistently has a large influence (in magnitude) on alignment to lineage centroids in the LCL embedding.

D.11. Additional details about applying other methods when identifying pathways related to fate commitment.

- **Usage of VariancePartition:** We apply the VariancePartition analysis (Hoffman & Schadt, 2016) based on the `variancePartition` R package (found on Bioconductor), which uses a mixed-effects model to model the amount of variance explained in each gene’s expression by each covariate. We use a regression form of `(1 | Lineage_label) + (1 | Cell_type) + (1 | Time_point)` and the `variancePartition::fitExtractVarPartModel` function.

- **Usage of GEMLI:** We apply GEMLI (Eisele et al., 2024), which is modified from `predict_lineages` function in the GEMLI R package (found on GitHub, UPSUTER/GEMLI). The authors design this function to take the normalized single-cell gene expression matrix by default and to compute “marker genes,” which are the only features used in the remainder of the function when predicting whether two cells are from the same lineage. However, we code a modification of `predict_lineages` that allows us to avoid computing marker genes (and instead use all the provided features), since we want to apply GEMLI to LCL or scVI coordinates. Otherwise, we use the default GEMLI parameters. Specifically, we use `fast=TRUE`, `repetitions=100`, and `sample_size=2/3`. We set `desired_cluster_size=c(50,200)` since our application of GEMLI on the CellTag-multi dataset focuses on only the cells in the largest 50 lineages.

Then, to predict, we use the `test_lineages` function with `valid_fam_sizes=c(50,200)` to compute the true and false positive rates.

- **Usage of CoSpar:** We apply CoSpar (Wang et al., 2022), which is the `cospar` Python package (found on GitHub at AllonKleinLab/cospar). We use the recommended parameters when using CoSpar on the CellTag-multi data. Specifically, we use

`cospar.tmap.infer_Tmap_from_multitime_clones` with the following parameters:

- `clonal_time_points=["3", "12", "21"]`,
- `later_time_point="21"`,
- `smooth_array=[15, 10, 5]`,
- `sparsity_threshold=0.2`,
- `intraclone_threshold=0.2`.

- **Usage of GSEA:** We perform GSEA using the `clusterProfiler` R package (Yu et al., 2012). Specifically, we use the `clusterProfiler::gseGO` function using the recommended parameters in their tutorial, `minGSSize=10` and `maxGSSize=500`. We only analyze the biological process ontology (i.e., `ont="BP"`) using the pathways from the `org.Mm.eg.db` (for the mouse).

The inputs to GSEA are: 1) the integrated gradient of each gene (based on our description in the Section 3.4 in the main text), 2) the correlation between CoSpar’s fate bias for differentiating into either reprogramming/dead-end cell types and a normalized gene’s expression across those same cells, and 3) the VariancePartition’s variance explained among all the genes for the lineage label.

D.12. Additional details on the CellTag and CellTag-multi integration analysis

When analyzing the integration between the CellTag and CellTag-multi datasets, we first perform batch correction using Seurat’s anchor-based integration framework (Stuart et al., 2019; Hao et al., 2021) prior to applying LCL. This preprocessing step is necessary because the LCL architecture does not explicitly incorporate a batch-effect correction term, as such corrections are not naturally compatible with the contrastive learning objective.

To do this batch correction, we use Seurat’s anchoring method. Our empirical experiment demonstrates that this batch correction method is more suitable than using scANVI. The integration features are selected using `Seurat::SelectIntegrationFeatures()` (`nfeatures = 3000`), and cross-dataset anchors are identified using `Seurat::FindIntegrationAnchors()` (default CCA-based integration with PCA dimensions 1-30). The integrated expression assay is then constructed using `Seurat::IntegrateData()` (default `new.assay.name = "integrated"`).

E. Additional results

In Section E.1, we provide additional results using VariancePartition. In Section E.2, we provide additional results using LCL, scVI, or supervised UMAP for the analyses on either the LARRY and CellTag datasets. In Section E.3, we provide additional GEMLI and GSEA evaluations on the full LARRY dataset. In Section E.4, we provide additional results about the KNN classifier or compositional analysis for both the LARRY and CellTag datasets. In Section E.5, we report additional results for the melanoma sequential-treatment dataset. In Section E.6, we provide additional results on the CellTag-multi

and CellTag integrated analysis. In Section E.7, we document the runtime for each analysis, provide guidance on how the hyperparameters impact the runtime, and report the hyperparameters used for each analysis.

E.1. Additional results about the VariancePartition analysis.

In the main text, we use VariancePartition (Hoffman & Schadt, 2016) to analyze how much of each gene’s expression can be explained by each covariate (lineage label, time point, and cell type). We demonstrate this on the LARRY dataset (Weinreb et al., 2020) in the main text (Figure 3). Here, we display the VariancePartition results for the CellTag and CellTag-multi dataset (Figure 13). We can see that the amount of variance explained by lineage is: (1) highly variable from gene to gene, and (2), on average, is relatively small when compared to cell type and time points. These results further emphasize why learning fate-commitment dynamics from scLT data is difficult – the signal from the lineage labels is not the dominant source of biological information. Unlike other machine learning tasks, where the “largest source of signal” is typically the primary signal of interest, we are instead interested in the biological relevance of secondary signals.

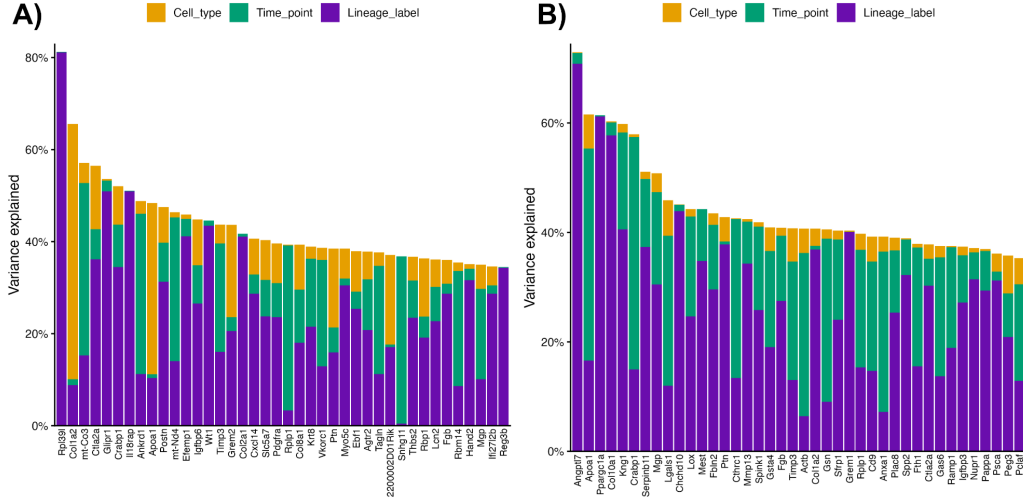


Figure 13. The VariancePartition results for CellTag (A) and CellTag-multi (B), showing the top 40 genes in each respective dataset. The plot is organized similarly to Figure 3 in the main text.

E.2. Additional results of LARRY and CellTag datasets

Figure 14 shows the LCL embedding for the CellTag dataset. This mirrors Figure 6 in the main text.

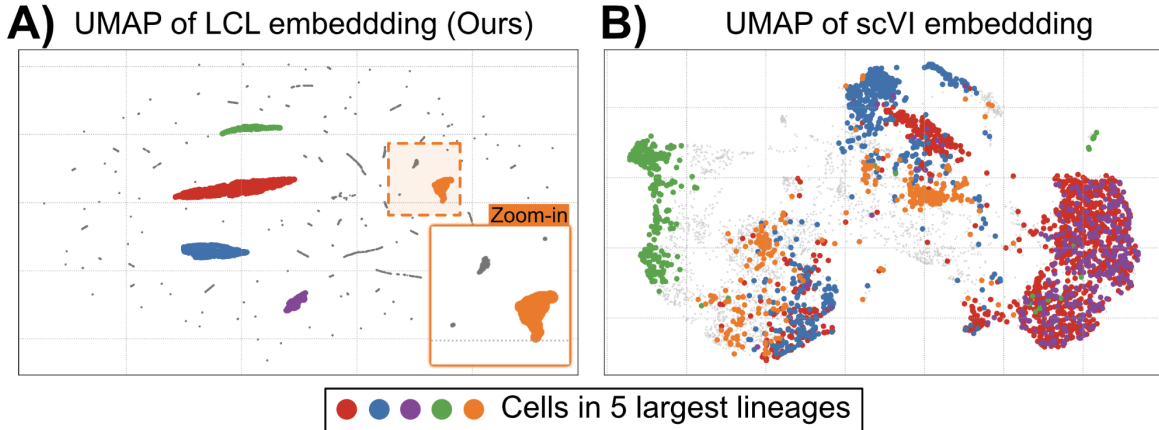


Figure 14. UMAP of the LCL embedding (A) or the scVI embedding (B) for the CellTag dataset. The cells in the largest five lineages are highlighted with a different color in their respective datasets.

Figure 15 shows the Supervised UMAP of both the CellTag and LARRY datasets. These plots contrast the embeddings of LCL and scVI in Figure 6 in the main text. We note that Figure 6 in the main text shows the LCL embedding specifically for the “LARRY” (i.e., with all 41,093 cells) dataset.

Figure 16 shows the UMAP of the LCL embedding of “LARRY (Largest 200)” The embedding also looks “island-like” in the UMAP, illustrating that LCL has found a way to separate many lineages and to cluster cells within the same lineages, even if the lineage contains cells of many different cell types collected at different time points.

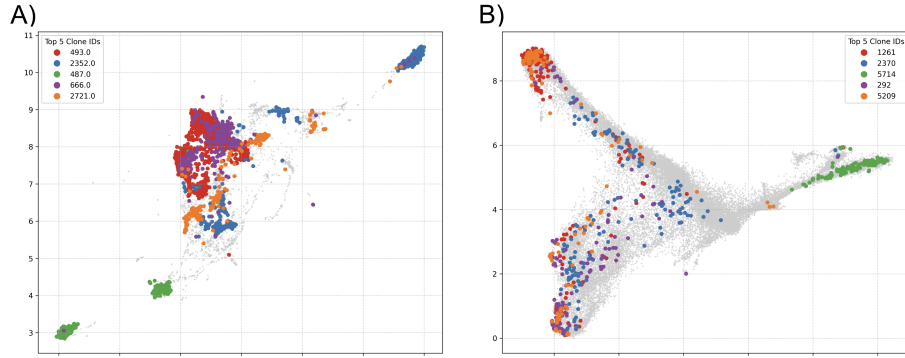


Figure 15. **A)** Supervised UMAP of the CellTag dataset. **B)** Supervised UMAP of the LARRY dataset. The cells in the five largest lineages are highlighted in different colors. These colors are the same as those shown in Figure 6 in the main text.

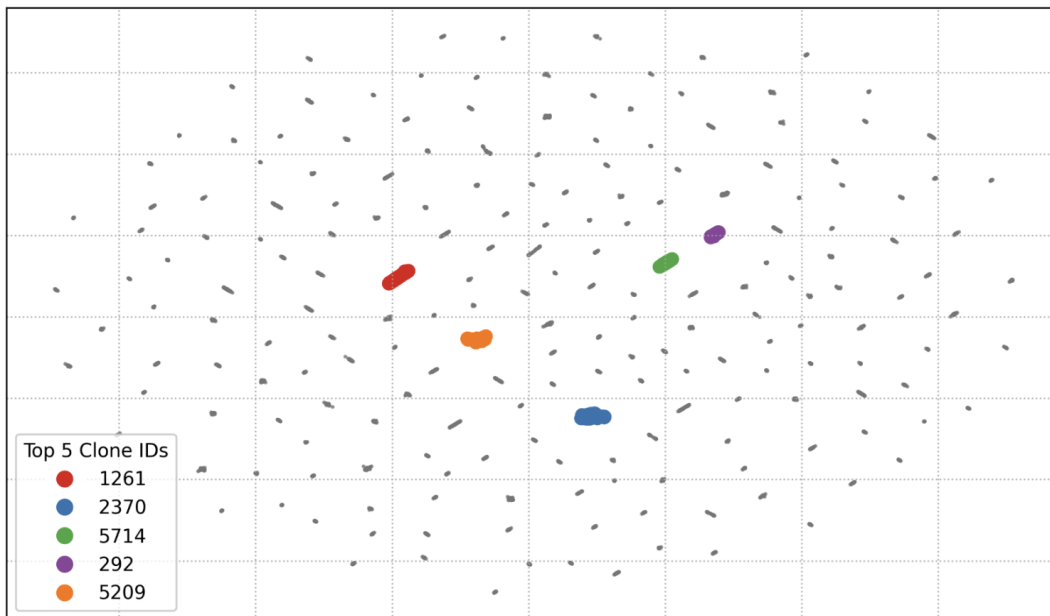


Figure 16. UMAP of the LCL embedding of “LARRY, Largest 200.”

E.3. Additional GEMLI and GSEA evaluation on the full LARRY dataset

The full LARRY dataset is highly heterogeneous, tracking 41,093 cells across 2,813 distinct lineages in a complex hematopoietic differentiation system. To further evaluate whether LCL learns a representation that preserves lineage structure in this heterogeneous setting, we apply GEMLI (Eisele et al., 2024) to compare whether different embeddings can predict if two cells originate from the same lineage. Using the LCL embedding substantially improved GEMLI’s predictive performance, achieving an AUROC of 0.73, compared to 0.13 using the scVI embedding and 0.16 using log-normalized gene expression features (Figure 17).

We further evaluate whether the LCL embedding supports pathway-level discovery of lineage-associated biological signals. We apply Hotspot-based gene scoring followed by GSEA to compare pathway enrichment across methods. LCL uniquely identified statistically significant fate-associated pathways, including *response to external stimulus* and *oxygen-containing compound*, that are not detected using scVI or CoSpar (Figure 18). These results support the conclusion that LCL improves

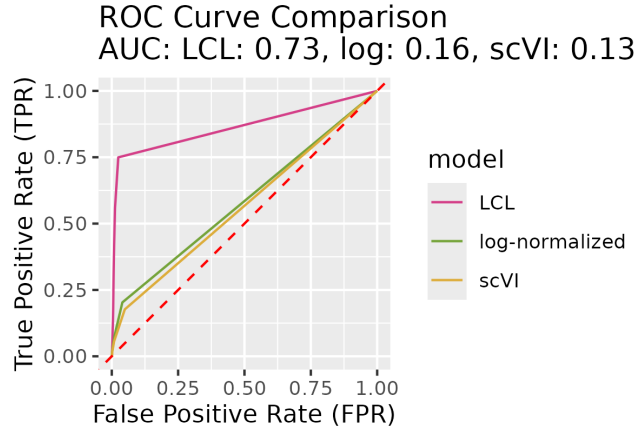


Figure 17. ROC curves for GEMLI lineage-pair prediction on the full LARRY dataset using different input representations. GEMLI predicts whether two cells originate from the same lineage using a random forest classifier. LCL achieves an AUROC of 0.73, outperforming scVI and log-normalized gene-expression inputs.

both lineage-structure recovery and downstream biological discovery on the full LARRY dataset.

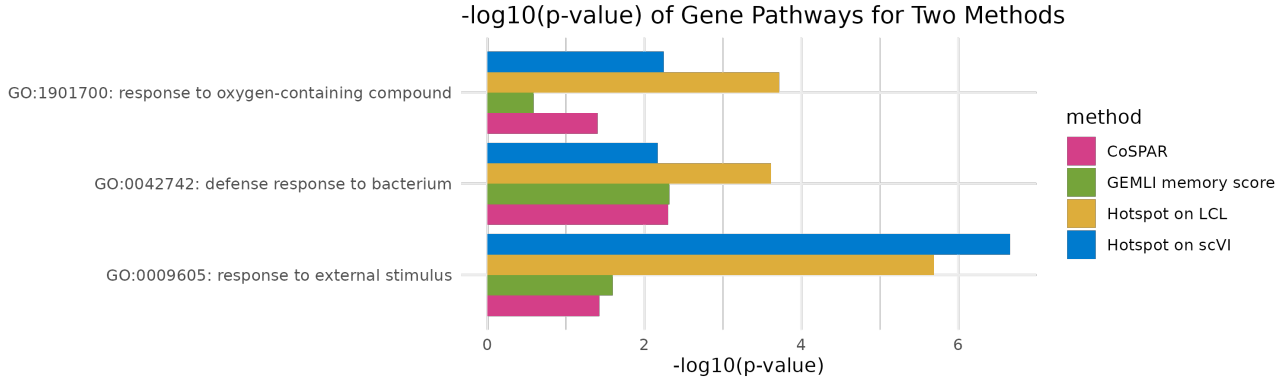


Figure 18. Pathway enrichment comparison on the full LARRY dataset using Hotspot-based gene scoring followed by GSEA. The plot compares pathway enrichment across methods using $-\log_{10}(p\text{-value})$. LCL uniquely identifies statistically significant pathways such as response to external stimulus and oxygen-containing compound, which are missed by alternative methods.

E.4. Additional results of KNN and compositional analysis

Evaluating lineage clusters and predicting lineage. Table 12 shows the CH Index for evaluating if a lineage is clustered together in the embedding and the KNN accuracy for predicting a cell’s lineage using the KNN classifier with five nearest neighbors (“KNN(5)”). We show these metrics on both the training and test sets. These correspond to the results in Table 1 in the main text. Specifically, the results in Table 1 correspond to “CellTag” and “LARRY, Largest 200” in Table 12. Recall that a higher CH Index and a higher KNN accuracy indicate that the embedding generalizes better. We bold the highest training and testing values for each dataset in Table 12.

Additional neural embedding baselines. We additionally evaluate embeddings trained using cross-entropy and triplet loss. Table 13 reports their KNN lineage-prediction error and KL divergence for future cell-type composition prediction on held-out cells. Lower values indicate better performance for both metrics.

Table 12. CH Index and KNN classifier’s accuracy for predicting lineage. The values in the “KNN(5) Test” match the values in Table 1 in the main text (since Table 1 shows error, and this table shows accuracy)

Dataset	Method	CH Index		KNN(5)	
		Train	Test	Train	Test
CellTag	LCL	15351.3	14.7	98.98%	54.06%
	supUMAP	66.1	11.6	71.8%	37.2%
	scVI	18.1	4.9	49.7%	47.1%
LARRY, Largest 200	LCL	56335.9	9.3	99.0%	13.6%
	supUMAP	125.4	6.2	33.7%	5.9%
	scVI	30.4	4.8	12.6%	10.8%
LARRY	LCL	430.1	7.6	47.9%	2.5%
	supUMAP	35.9	5.3	3.9%	1.6%
	scVI	12.2	3.2	4.3%	3.3%

Table 13. Performance of the additional neural embedding baselines on held-out lineage prediction and future cell-type composition prediction. Lower values indicate better performance.

Dataset	Method	KNN Test Error ↓	KL Divergence ↓
LARRY (Largest 200)	Cross-entropy (CE)	0.898	0.796
	Triplet loss	0.905	0.688
CellTag	Cross-entropy (CE)	0.437	0.771
	Triplet loss	0.530	0.606

Generalizing to unseen lineages in the LARRY and CellTag datasets. We also consider a more challenging approach to constructing a test set, in which the test set cells belong to lineages not present in the training set (i.e., “unseen”). Since the lineages in the training and test sets are mutually exclusive, this experiment assesses a more challenging generalization. We use a different evaluation metric for this setting. Based on the fitted LCL model, we construct a KNN graph of the test cell’s embedding and count the number of test-cell nearest neighbors that are from the same lineage. We are interested in this experiment since, if successful, LCL would be applied to biological systems where *in vitro* lineage barcoding is not possible. Our results demonstrate that LCL offers a modest improvement over scVI and a more substantial improvement over Supervised UMAP.

Table 14 shows the performance of each of the three embedding methods in the “unseen lineages” scenario whereby 10% of the lineages are entirely withheld from the training set, and the cells of these withheld lineages formed the test set. This setting is more difficult because the lineages in the training and test sets do not overlap.

Since the lineages between the training and test sets do not overlap, we use a different evaluation metric. Specifically, the values reported in Table 14 refer to the average percentage of nearest-neighbor test cells (among the five nearest neighbors) that share the same lineage as the test cell in question (“KNN(5)”). A higher number here indicates that the embedding generalizes to unseen lineages, with cells in the same test lineage mapped to similar coordinates in the embedding space. We bold the highest values for each dataset in Table 14.

E.5. Additional results on the melanoma sequential-treatment dataset

We evaluate whether LCL generalizes to a cancer system with treatment-driven clonal fate dynamics using the melanoma sequential-treatment dataset from Schaff et al. (Schaff et al., 2024). As summarized in Table 15, LCL outperforms the corresponding baselines for both KNN lineage recovery and KL-divergence prediction of future survivor composition.

The lower KNN lineage error indicates that the LCL embedding captures clone-specific structure in held-out cells, while the reduced KL divergence suggests that first-stage lineage-aware embeddings retain information predictive of future treatment-response compositions. These results extend our evaluation beyond developmental and reprogramming systems, supporting the applicability of LCL to a cancer model with distinct lineage dynamics and sequential treatment perturbations.

Table 14. Percentage of nearest-neighbor test cells with the same lineage for unseen lineages

Dataset	Method	KNN(5)
CellTag	LCL	44.90%
	supUMAP	37.20%
	scVI	45.70%
LARRY, Largest 200	LCL	50.60%
	supUMAP	41.40%
	scVI	48.60%
LARRY	LCL	26.90%
	supUMAP	16.40%
	scVI	21.30%

Table 15. Performance of LCL on the melanoma sequential-treatment dataset. Lower values indicate better performance.

Evaluation Metric	LCL	Baseline
KNN Lineage Error	0.746	0.986
KL Divergence for Future Survivor Composition	1.047	1.689

E.6. Additional results on CellTag/CellTag-multi integration

Batch correction. Before applying LCL to the experiment assessing whether the lineage labels in CellTag-multi are informative for learning fate dynamics in CellTag, we need to perform batch correction. Figure 19A shows a clear batch effect between the two datasets before any correction. Since LCL does not do a batch correction, Figure 19 shows the resulting UMAP, colored by dataset identity, illustrating an effective batch-correction, standard-anchor-based integration workflow. This integration of gene expression vectors for both datasets provides a common coordinate system on which LCL can be applied to investigate lineage-prediction transfer between related datasets.

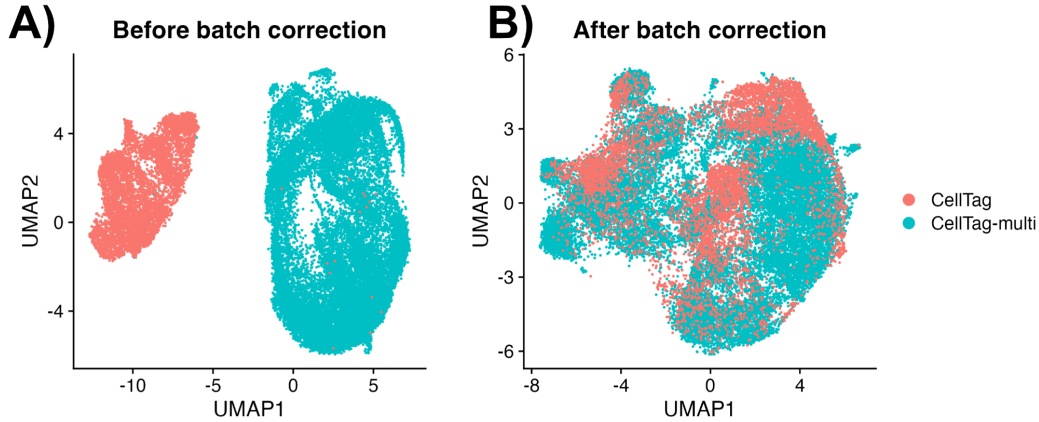


Figure 19. **A)** Gene expression of the CellTag and CellTag-multi datasets, where the two datasets are clearly separable due to batch effects. Cells are colored by dataset identity. **B)** Baseline Seurat batch-correction integrating both datasets together.

We note that, even after integrating the two datasets, the lineages in CellTag are not necessarily co-localized. In Figure 20, we see that the largest 5 lineages in CellTag are quite dispersed throughout the integrated gene expression space.

Size-normalized KNN lineage enrichment (modified KNN). While the CH Index captures *global* separation of lineage clusters, it may fail to reflect settings in which lineage structure is expressed primarily at a local scale. Moreover, a standard KNN classification accuracy is ill-suited to this integration setting, as it is highly sensitive to lineage-size imbalance and does not provide a meaningful baseline under random mixing. We therefore define a size-normalized neighborhood enrichment metric that directly quantifies whether same-lineage cells are overrepresented in local neighborhoods relative to their global frequency.

Fix a neighborhood size K and let $\mathcal{N}_K(i)$ denote the set of K nearest neighbors of cell i in the embedding space (excluding

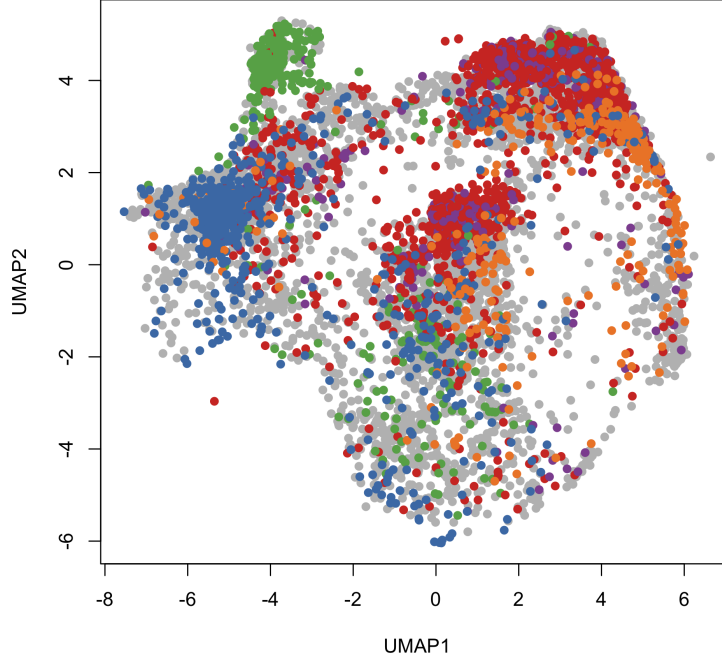


Figure 20. The largest 5 lineages of the CellTag dataset in the Seurat batch-corrected embedding space. This plot illustrates the need to run LCL after batch correction, since the true lineages (which LCL will not have access to during training, since CellTag will be treated as an "unlabeled" dataset) are not co-localized in this embedding space.

i itself). For each cell i with lineage label y_i , we compute

$$p_{\text{local}}(i) = \frac{1}{K} \sum_{j \in \mathcal{N}_K(i)} \mathbb{1}[y_j = y_i], \quad p_{\text{global}}(i) = \frac{n_{y_i} - 1}{n - 1},$$

where n_{y_i} is the size of lineage y_i among the n evaluated cells. Here, $p_{\text{global}}(i)$ represents the expected fraction of same-lineage neighbors under random mixing, accounting for lineage size.

We define the per-cell enrichment score as

$$\text{Enrich}(i) = \frac{p_{\text{local}}(i)}{p_{\text{global}}(i)}.$$

An enrichment value greater than 1 indicates that cell i 's neighborhood contains more same-lineage cells than expected by chance, while a value near 1 corresponds to no lineage-specific enrichment. We summarize embedding quality using the mean and median enrichment across cells:

$$\text{Enrich}_{\text{mean}} = \frac{1}{n} \sum_{i=1}^n \text{Enrich}(i), \quad \text{Enrich}_{\text{median}} = \text{median}_i \text{Enrich}(i).$$

Larger enrichment scores indicate stronger local lineage coherence. We report results with neighborhood sizes of 50 to assess robustness across spatial scales.

E.6.1. INTEGRATION HYPERPARAMETER DIAGNOSTICS: CALINSKI–HARABASZ AND SIZE-NORMALIZED KNN ENRICHMENT

To summarize integration quality across different semi-supervised LCL settings (e.g., varying λ and the number of unlabeled cells U), we evaluate each fitted embedding using two complementary diagnostics computed on the CellTag cells (treated as

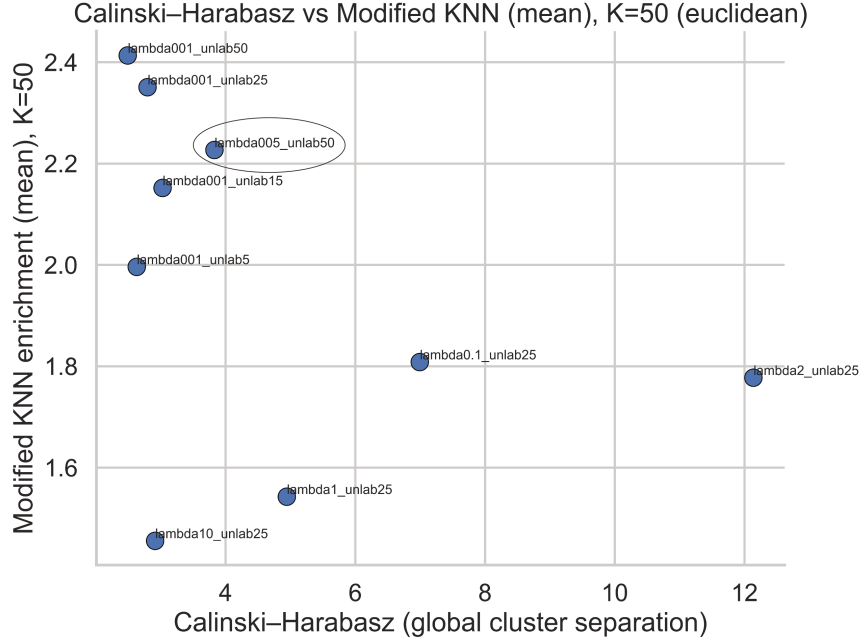


Figure 21. Trade-off between global lineage separation (CH Index) and local lineage coherence (size-normalized KNN enrichment, $K = 50$) across LCL integration settings. The circled point denotes the selected hyperparameter setting used in downstream analyses.

unlabeled during training, but with lineage barcodes available for post hoc evaluation). These metrics capture two distinct desiderata: (i) whether lineage structure forms globally separated clusters, and (ii) whether local neighborhoods are enriched for same-lineage cells beyond what would be expected from lineage size alone. As Figure 21 shows, we choose the setting with $\lambda = 0.05$ and $U = 50$ that balances strong local lineage enrichment with moderate global cluster separation, avoiding over-fragmentation while preserving lineage coherence.

E.6.2. MARKER GENES OF LCL CLUSTERS.

Continuing the analysis of the genes that strongly correlate with fate dynamics (Table 4 in the main text), we first show in Figure 23 the Leiden clustering of the CellTag (unlabeled) cells in the learned LCL embedding. We use the `Seurat::FindClusters` function, with `resolution=0.1`, to learn these 8 clusters. These clusters provide an automated way to separate the large “islands” in the LCL embedding. Afterwards, we use a Wilcoxon test to learn the differentially expressed genes unique to each island using a one-versus-rest strategy used by the `Seurat::FindAllMarkers` function. We filter the list of genes such that: 1) the log fold change of the gene (in the target cluster, relative to all the other cells) is 3 or larger, 2) the percent of cells expressing the gene in the target cluster is at least 30%, and 3) the adjusted p-value is less than 0.05. Only 17 genes remain after this stringent filtering, of which we focus on *Ptn*, *Sfrp1*, and *Shh*, three genes strongly associated with whether a fibroblast cell will successfully reprogram into an iEP.

The genes identified through LCL clustering (*Ptn*, *Sfrp1*, and *Shh*) represent critical molecular divergence points between successful and unsuccessful reprogramming trajectories. *Ptn* (Pleiotrophin) and *Sfrp1* (Secreted Frizzled Related Protein 1) are established markers of “dead-end” cells that fail to transition into Induced Endoderm Progenitors (iEPs) (Jindal et al., 2024). *Sfrp1* is a known antagonist of Wnt signaling, and its upregulation is part of a “dead-end” signature that reinforces the fibroblast identity or leads to off-target states. In contrast, the activation of the *Shh* (Sonic Hedgehog) signaling pathway is associated with successful reprogramming into the endoderm lineage. While the CellTag-multi (Jindal et al., 2024) study focuses heavily on the Wnt/TGF- β axis to distinguish on-target iEP trajectories from dead-end fates, *Shh* is a documented driver of definitive endoderm differentiation and embryonic gut development, serving as a hallmark of successful iEP maturation (Guzzetta et al., 2020).

We illustrate the co-localization of these key genes in Figure 24, where we contrast the original integrated gene expressions against the LCL embeddings. Specifically, we observe in Figure 24B that genes like *Ptn* and *Sfrp1* co-localize very strongly into certain LCL “islands,” which consist of mainly fibroblasts that do not successfully reprogram into iEPs. This reinforces

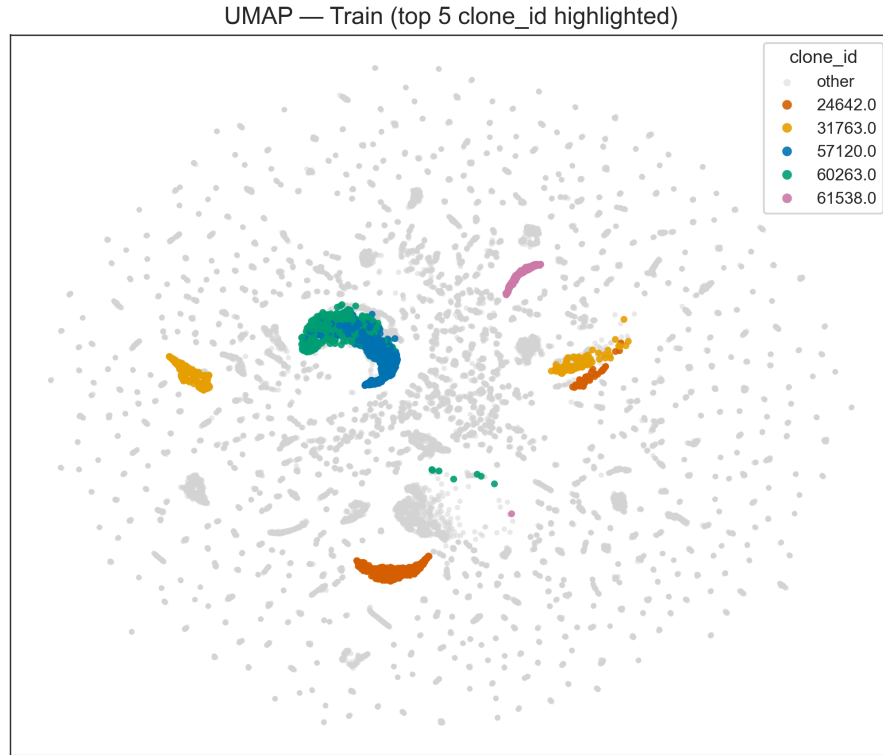


Figure 22. UMAP visualization of the LCL embedding for the CellTag-multi training set, with the five largest lineages highlighted. Cells belonging to the top five lineages (by clone size) are colored, while all other cells are shown in gray. This visualization illustrates the within-lineage coherence and spatial separation learned by LCL on the labeled reference dataset.

LCL’s strength: LCL can isolate the gene expression component that specifically relates to fate dynamics encoded by lineage labels in the CellTag dataset, *even though* LCL did not access or use these lineage labels during training. This demonstrates both LCL’s generalizability and potential to advance cell biology.

E.7. Additional results on runtime performance

We discuss how LCL scales with different dataset sizes, batch configurations, and hyperparameters. This provides additional insight into the computational feasibility of applying LCL to other large-scale single-cell lineage-tracing datasets. The main factors influencing runtime are the number of cells, mean number of cells per lineage, batch size N , size factor α , and number of training epochs. Table 16 shows the number of cells, number of cells per lineage, the hyperparameters we use for LCL, and the resulting run time of the LCL fitting procedure.

- **Effect of Number of Cells and Cells per Lineage:** There is a moderate positive correlation between both the number of cells ($r = 0.59$) and the number of cells per lineage ($r = 0.55$) with the runtime. This suggests that datasets with more cells or more cells per lineage tend to require longer runtimes. For instance, the dataset with 41,093 cells and an average of 33 cells per lineage takes 1.78 hours to run.
- **Effect of Batch Size and Positive Pairs:** Larger datasets with more positive pairs take longer to process, especially when the size factor is increased.
- **Impact of Hyperparameters:** The batch size and size factor directly influence runtime, with larger batch sizes reducing the number of total batches and slightly improving GPU utilization. The dataset’s number of epochs and complexity (e.g., simulation datasets with varying beta values) also contribute significantly to runtime variability.
- **Efficiency on Google Colab:** Training the model on Google Colab’s NVIDIA L4 GPU is efficient for medium-sized datasets (up to 1.7 million pairs), with average run times of less than 2 hours. The 2.3 million trainable parameters and efficient use of GPU and system resources allow LCL to handle various datasets without exceeding resource limits.

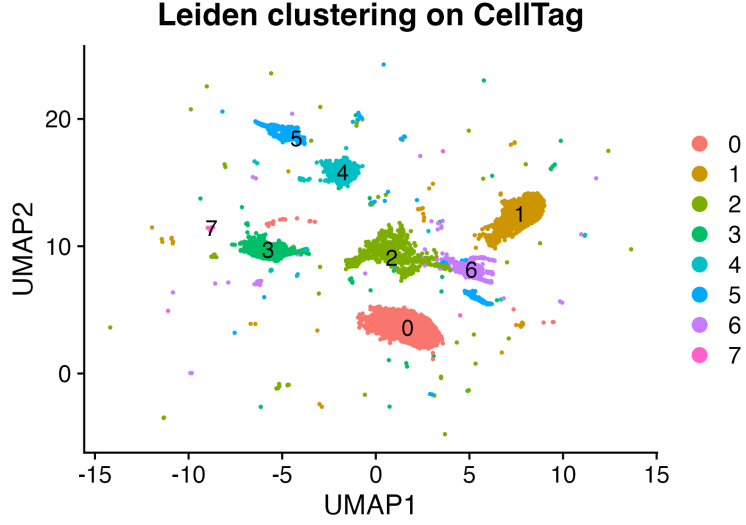


Figure 23. Leiden clustering of the CellTag dataset’s LCL embedding.

Table 16. Dataset and Hyperparameter Configurations with Runtimes

Dataset	Num. cells	Num. cells per lineage	Hyper-parameters ($N, \alpha, U, \lambda, \tau$)	Runtime (hours)
CellTag	5893	34	50, 0.04, 5, 0.01, 0.5	1.1
CellTag-multi	22238	16	250, 0.02, 50, 0.01, 0.5	1.3
CellTag(multi) Integration	28131	19	250, 0.02, 50, 0.05, 0.5	1.3
LARRY, Largest 200	10148	50	150, 0.5, 15, 0.01, 0.5	1.02
LARRY	41093	14	260, 0.45, 15, 0.01, 0.5	1.78
Pseudo-real, $\beta = 0.1$	41093	33	200, 0.5, 0, 0, 0.5	1.36
Pseudo-real, $\beta = 0.3$	41093	42	150, 0.4, 0, 0, 0.5	1.09
Pseudo-real, $\beta = 0.5$	41093	51	100, 0.4, 0, 0, 0.5	1.62
Pseudo-real, $\beta = 0.7$	41093	68	100, 0.4, 0, 0, 0.5	2.04
Pseudo-real, $\beta = 0.9$	41093	106	100, 0.35, 0, 0, 0.5	2.15

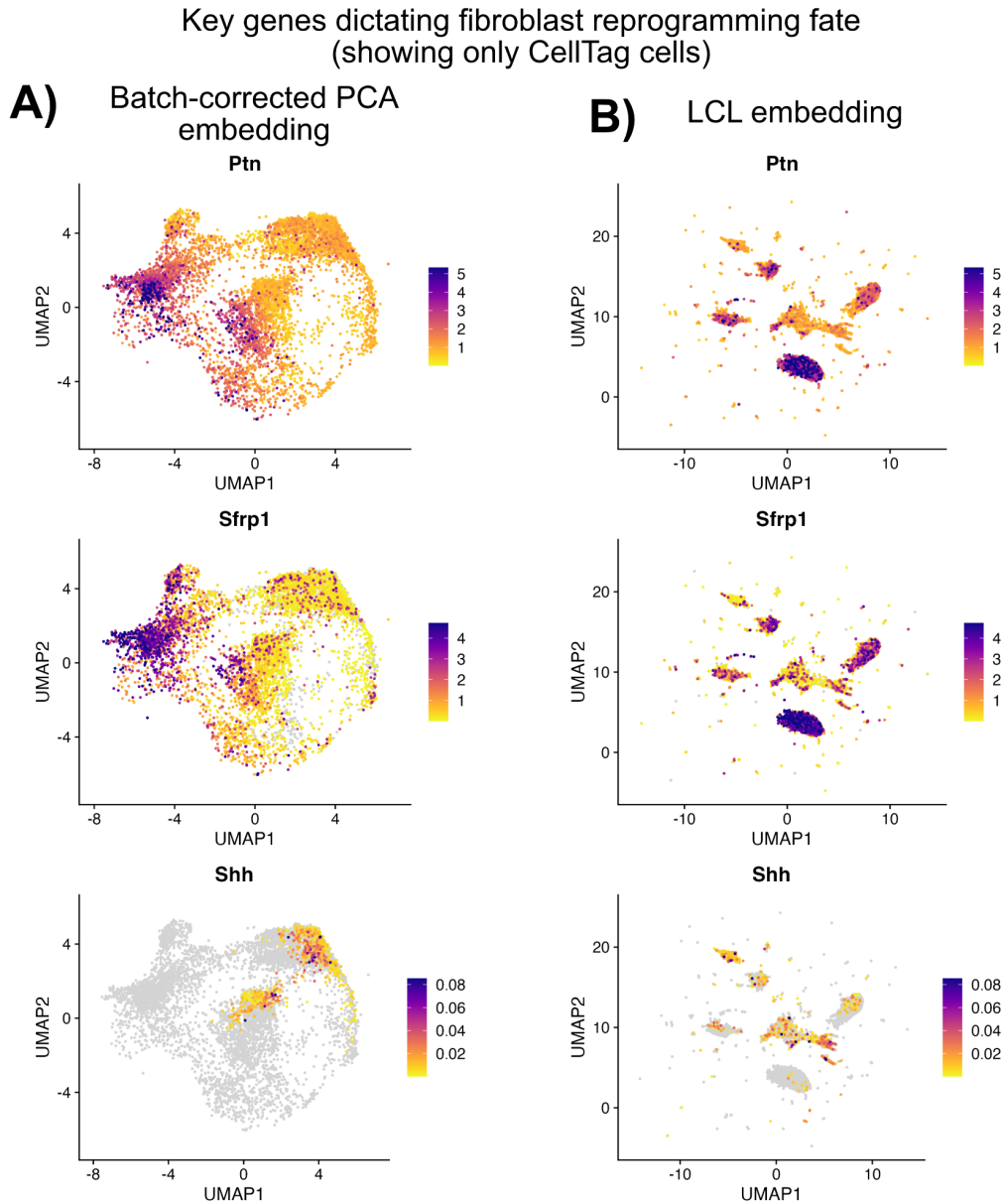


Figure 24. Key genes using the batch-corrected (original) embedding (A), compared to the LCL embedding (B). We observe more overt co-localization of key genes in the LCL embedding, illustrating its ability to cluster cells by lineage so that cells with similar fate dynamics share similar LCL coordinates.